

Министерство науки и высшего образования РФ

Московский авиационный институт
(национальный исследовательский университет)

П.В. НОВИКОВ

**ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ
ПРОГРАММИРОВАНИЕ
на C++ для Visual Studio 2019/22**

Учебное пособие к лабораторным работам

Утверждено
на заседании редсовета
ноября 2024 г.

Москва
Издательство МАИ
2025

УДК 681.3.06

Новиков П.В. Объектно-ориентированное программирование на C++ для Visual Studio 2019/22: Учебное пособие. / Под редакцией доцента, кандидата техн. наук Кима Р.В. – М.: Изд-во МАИ, 2025. 96с.

Представлены базовые приёмы объектно-ориентированного программирования на алгоритмическом языке C++ в версии Visual Studio C++ 2019/2022. Рассмотрены теоретические понятия и синтаксические конструкции, соответствующие объектно-ориентированной парадигме. Приведены примеры программ на разработку классов и создание объектов, на применение механизма наследования классов, на использование полиморфных (переопределённых и перегруженных) методов. Показаны особенности виртуальных функций и их преимущества. Изучены приёмы программирования с использованием абстрактных классов и интерфейсов, а также приёмы композиции классов и объектов на примерах графических объектов. Представлены примеры программ с полиморфными объектами. и с динамическими объектами. Дополнительно исследованы объекты как входные и выходные параметры функций - членов класса (методов), и самостоятельных функций. Показаны технологические приёмы создания объектно-ориентированных программ на Visual Studio C++ 2019/2022. Предложены задания по всем изучаемым разделам.

Пособие предназначено для студентов вузов, обучающихся по направлениям 09.03.01 «Информатика и вычислительная техника» и 09.03.04 «Программная инженерия».

Рецензенты:

кафедра (зав. кафедрой);

канд. техн. наук,

ISBN 978-5-4316-0625-0

© Московский авиационный институт
(национальный исследовательский
университет), 2025

ПРЕДИСЛОВИЕ

Учебное пособие ориентировано на обучение студентов направления «Программная инженерия», для специальности «Вычислительные машины, системы и сети», а также «Программное обеспечение вычислительной техники, автоматизированных систем, комплексов и сетей». Представлен цикл лабораторных работ по дисциплине «Объектно-ориентированное программирование». Содержание этой дисциплины опирается на полученные знания, умения и навыки при освоении императивного (процедурного) программирования на языке C++, получаемые в процессе изучения дисциплины «Программирование». Объектно-ориентированную парадигму предлагается осваивать при выполнении восьми лабораторных работ (32 учебных часа). Вводятся понятия: объект, класс (объектный тип данных), переменная - члена класса (поле данных), функция - члена класса (метод), конструктор, деструктор, метод-акцессор, открытые (public), закрытые (private) и защищённые (protected) компоненты класса, виртуальная функция (virtual), абстрактный методы (чисто виртуальная функция), абстрактный класс, интерфейс (interface), динамический объект и полиморфный объект. Рассматриваются основные способы синтеза объектно-ориентированных программ: инкапсуляция, наследование, полиморфизмы методов (перегрузка overload и переопределение override), агрегация (композиция классов и объектов), использование объектов как входных и выходных параметров функций.

Среда Visual Studio C++ 2019/2022 позволяет создавать на языке программирования C++ многофайловые программные решения в виде единых проектов, что позволяет считать эту среду программирования ориентированной на разработку современных программных продуктов.

Общий порядок выполнения лабораторных работ

1. Прочитать тему и цель, изучить общие сведения к каждой работе.
2. Освоить представленные и описанные в материалах к этой лабораторной работе объектно-ориентированные конструкции, используя примеры.
3. Разработать объектно-ориентированную программу в соответствии с вариантом задания, применяя новые синтаксические конструкции и новые приёмы программирования.
4. Предъявить правильно функционирующую программу преподавателю, предварительно убедившись в правильности её работы. В случае ошибочности работы программы, отладить её, найдя синтаксические и алгоритмические ошибки и исправив эти ошибки.
5. После того как выполнение работы зачтено преподавателем, рекомендуется распечатать текст программы и результаты её работы.
6. Оригинальные файлы с текстами программ следует сохранить на личном носителе для оформления отчёта, после чего обязательно удалить их с жёсткого диска стационарного компьютера в компьютерном классе.
7. Оформив отчёт, защитить лабораторную работу у преподавателя.

Все задания должны быть выполнены среде Visual Studio C++ в версиях не ранее, чем Visual Studio 2019/2022. Для каждого студента допускается использовать при выполнении работ его личный персональный компьютер.

Содержание отчёта по лабораторной работе

1. Титульный лист
2. Тема работы.
3. Цель работы.
4. Индивидуальное задание на выполнение.
5. Проекты создаваемых классов.
6. Текст объектно-ориентированной программы.
7. Результаты работы программы.
8. Выводы на основе выполненной работы.
9. Список использованных источников (литература).

РАБОТА № 1. ОБЪЕКТ И КЛАСС. ИНКАПСУЛЯЦИЯ

Цель работы – знакомство с понятиями объект и класс, переменные - члены класса (поля данных) и функции, члены класса (методы). Изучение терминов: открытые (public) и закрытые (private) компоненты объектов и классов. Создание методов-конструкторов и методов-акцессоров. Освоение технологии разработки классов и создания объектов в проектах консольных приложений на языке программирования C++ для Visual Studio 2019/2022.

Общие сведения

Объект – это *отдельный независимый программный компонент*, имеющий собственные индивидуальные характеристики (называемые **переменные-члены** или **поля объекта**) и наделённый своим оригинальным поведением (задаваемым **функциями-членами** или **методами объекта**). Поля объекта и методы объекта привязаны к данному объекту и могут быть использованы только в контексте взаимодействия с ним. Объект есть результат соединения в единое целое структурированной переменной и функций, обрабатывающих поля этой структуры. Соединение в объектах полей структур и методов обработки этих полей называют **инкапсуляцией**. В языке C++ и во многих других языках программирования инкапсуляция предполагает контроль доступа к каждому полю объекта и к каждому методу объекта. Объект есть переменная типа данных, представляющего собой особый, *объектный тип данных*.

Для создания объектов в C++ необходимо разрабатывать особые объектные типы данных – **классы**. Объявление объектного типа данных в C++ начинается с ключевого слова *class*. Другими словами, объект есть переменная объектного типа данных, также называемая *экземпляр класса*. В классе объявляются поля (их имена, типы и доступ к ним) и методы (их имена, типы входных аргументов и тип возвращаемого значения, а также доступ к методам). В связи с этим, полное название поля объекта в C++ звучит как *переменная-член класса*, а полное название метода объекта в C++ звучит как *функция-член класса*. Однако чаще в C++ используются более короткие термины: переменная-член и функция-член, названные выше.

В отличие от обычных пользовательских функций языка C++, создаваемых перед основной программой (перед функцией main), либо объявляемых перед функцией main, а создаваемых после функции main (которые были изучены ранее в рамках дисциплины «Программирование»), функции-члены (методы), не вызываются сами по себе. Если обычная функция в C++ может быть вызвана путём указания её имени с заданными аргументами (например, sin(0.5) или exp(x) и т.п.), то вызов каждого метода (каждой функции-члена) связан с конкретным объектом и является обращением к объекту (или посылкой сообщения к объекту). Например, если объект называется Obj, а метод Meth(), то на C++ пишут Obj.Meth(), что означает: к объекту Obj обращаются с сообщением Meth(). Объектно-ориентированная технология программирования предполагает синтез

классов, создание объектов и обмен сообщениями между объектами (посылку сообщений объектам).

Прямое обращение к переменным-членам (к полям объектов) не считается рациональным приёмом. Поля объекта рекомендуется закрывать, то есть делать доступными только для своих собственных методов. Это весьма характерно для инкапсуляции. *Закрытыми* могут быть не только поля объекта, но и сами методы. Такие закрытые методы вызывают из других методов этого же класса. Они не предназначены для посылки сообщений объектам. При этом важно знать, что в теле любого метода (функции-члена) можно обращаться и к открытым, и к закрытым методам и полям (переменным-членам) этого же класса. Внутри одного и того же класса все компоненты этого класса доступны друг для друга.

В языке C++ закрытые поля и закрытые методы объявляются в закрытых секциях класса. В начале закрытой секции класса пишут **private**:

Открытые поля и открытые методы в языке C++ объявляют в открытых секциях класса. В начале открытой секции класса пишут **public**:

В языке C++ количество открытых и закрытых секций в одном классе не ограничено. Секции располагаются в теле класса друг за другом в любом порядке.

Часто для сокращения исходного кода программы используют *правила умолчания*. В C++ для Visual Studio 2019/2022 все переменные-члены (поля) и функции-члены (методы) по умолчанию являются закрытыми (private). По этой причине рекомендуется всегда задавать доступ явно, чтобы отменить правила умолчания. Однако, при желании программиста, допускается использование правил умолчания для организации доступа к отдельным компонентам класса.

Особый метод для создания объектов называется **конструктор**. Этот метод осуществляет инициализацию полей объекта (присвоение начальных значений полям) при создании объектов. Он вызывается автоматически в момент создания объекта класса, но может быть вызван и явно. Также конструктор может выделять динамическую память во время работы программы при работе с динамическими полями. В языке C++ имя конструктора совпадает с именем его класса. Конструктор не имеет возвращаемого значения. При его объявлении никакой тип возвращаемого значения не указывается, в том числе void. В литературе по объектно-ориентированному программированию порой дают такое определение: «Конструктор класса — специальный блок инструкций, вызываемый при создании объекта».

Для реализации доступа к закрытым переменным-членам (полям объекта) создают специальные открытые методы, называемые методами-акцессоры (от английского слова access - доступ). Одни из таких методов возвращают значения закрытых полей объектов (*геттеры* от англ. Get), а другие — изменяют эти закрытые значения (*сеттеры* от англ. Set). Иногда в источниках акцессорами называют только методы-*геттеры*, а методы, задающие значение, тогда называют *модуляторы* или *мутаторы*). В C++

имена методов, реализующих доступ к закрытым полям объекта для чтения значений полей или для записи новых значений полей, могут быть любыми. Однако для удобства чтения программы имена этих методов, возвращающих значения закрытых полей, начинают со слова Get (GetX, GetY и т.п.). Имена методов, изменяющих значения закрытых полей, часто начинают со слов Set или Put (SetX, SetY, PutX, PutY и т.п.).

Объектно-ориентированная парадигма представляет собой набор общих принципов, стоящий выше конкретной реализации. При *объектно-ориентированном проектировании* разумно описывать компоненты будущего класса в виде, не связанном с синтаксисом языка.

В примере 1.1 дан проект класса Complex для работы с объектами - комплексными числами. Здесь предусмотрены закрытые вещественные поля для действительной и мнимой частей (Re и Im), открытые методы: конструктор Complex, акцессоры GetRe, GetIm, PutRe, PutIm, методы для печати комплексного числа в арифметической форме PrintComp и в тригонометрической форме PrintTrig, а также закрытые методы для амплитуды Amp и фазы Fi комплексного числа. Предусмотрен метод Sum для выполнения операции сложения двух комплексных чисел.

Пример 1.1. Проектирование класса Complex (комплексное число)

Имя класса: Complex

Поля данных (закрытые, вещественные):

Re – действительная часть;

Im – мнимая часть.

Методы:

1) Открытые методы

а) Конструктор, входные параметры, вещественные InitRe и InitIm – начальные значения полей Re и Im.

б) Методы-акцессоры

GetRe – возвращает вещественное значение поля данных Re,

GetIm – возвращает вещественное значение поля данных Im,

PutRe (новое Re) – задаёт новое вещественное значение для поля Re,

PutIm (новое Im) – задаёт новое вещественное значение для поля Im.

в) Прочие открытые методы

PrintComp – печатает комплексное число в алгебраической форме;

PrintTrig – печатает комплексное число в тригонометрической форме;

Sum(Complex) – возвращает комплексный результат сложения числа-объекта с числом-аргументом функции Sum;

2) Закрытые методы (вызываются в открытом методе PrintTrig)

Amp – возвращает вещественный модуль A комплексного числа,

$$A = \sqrt{\text{Re}^2 + \text{Im}^2}$$

Fi – возвращает вещественную фазу φ комплексного числа.

Пример 1.2. Разработка класса и создание объектов на языке C++

```
#include <iostream>
#include <windows.h>
#include <math.h>

using namespace std;

/* Объявление класса */
class Complex
{
private:    // закрытая секция
    double Re;
    double Im;
public:    // открытая секция
    Complex(double, double); // конструктор
    double GetRe();          // акцессоры - геттеры
    double GetIm();
    void PutRe(double);      // акцессоры - модуляторы
    void PutIm(double);
    void PrintComplex();     // печать комплексного числа
    void PrintTrig();
    Complex Sum(Complex);    // сложение объекта с аргументом
private:    // закрытая секция
    double Amp();
    double Fi();
};

Complex sum(Complex, Complex); // функция для сложения объектов

Complex::Complex(double Re, double Im)
{
    this->Re = Re;
    this->Im = Im;
}

Complex::Complex() {}

double Complex::GetRe() { return Re; }

double Complex::GetIm() { return Im; }

void Complex::PutRe(double Re) { this->Re = Re; }

void Complex::PutIm(double Im) { this->Im = Im; }
```



```

void Complex::PrintComplex() {cout<<" Re = "<<Re<<" Im = "<<Im<<endl;}

double Complex::Amp() { return sqrt(Re * Re + Im * Im); }

double Complex::Fi() { return atan2(Im, Re); }

void Complex::PrintTrig() {cout<<" A = "<<Amp()<<" Fi = "<<Fi()<<endl;}

Complex Complex::Sum(Complex B)
{
    Complex C(0, 0);
    C.PutRe(Re + B.GetRe());
    C.PutIm(Im + B.GetIm());
    return C;
};

Complex sum(Complex A, Complex B)
{
    Complex C(0, 0);
    C.PutRe(A.GetRe() + B.GetRe());
    C.PutIm(A.GetIm() + B.GetIm());
    return C;
};

int main()
{
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    system("color f0");
    Complex C1(3, 4), C2(5, 12), C3(0, 0);
    cout << "Результаты работы программы с классом Complex" << endl;
    C1.PrintComplex();
    C1.PrintTrig();
    C2.PrintComplex();
    C2.PrintTrig();
    C3 = sum(C1, C2); // сложение с помощью функции sum
    cout << endl << "C3:" << endl;
    C3.PrintComplex();
    C3.PrintTrig();
    C3 = C1.Sum(C2); // сложение с помощью метода Sum
    cout << endl << "C3:" << endl;
    C3.PrintComplex();
    C3.PrintTrig();
    system("Pause");
    return 0;
}

```

Рис. 1.1. Текст с результатами работы однофайлового проекта

Результаты работы программы с классом Complex

Re = 3 Im = 4

A = 5 Fi = 0.927295

Re = 5 Im = 12

A = 13 Fi = 1.17601

C3:

Re = 8 Im = 16

A = 17.8885 Fi = 1.10715

C3:

Re = 8 Im = 16

A = 17.8885 Fi = 1.10715

Для продолжения нажмите любую клавишу . . .

В примере 1.2 показана программа, реализующая на C++ проект класса Complex [10, 11]. Объекты (экземпляры) класса Complex - комплексные числа C1 и C2. Показано, как создают объекты и как обращаются к этим объектам с помощью функций-членов. Также показано обращение в теле функций-членов к закрытым полям (например, в теле методов-акцессоров) и показан вызов закрытых методов Amp и Fi в теле других методов этого же класса. В этом примере добавлена самостоятельная функция сложения двух комплексных чисел с именем sum для сравнения с методом Sum. Приведены результаты работы программы.

Из примера видно, что объектно-ориентированная программа на C++ состоит из трёх разделов: раздел объявления классов, раздел определения методов классов и раздел основной программы main, где используют объекты созданных классов. Язык C++ предоставляет программистам редкую и удобную возможность разделять объявление класса и определение методов класса, отсутствующую, например, в языках java или C#.

В разделе объявления классов каждый класс получает своё имя. Здесь объявляют открытые и закрытые секции, где присваивают полям имена и типы, а методам – имена, типы входных параметров и типы возвращаемых значений. Такая возможность позволяет программисту осуществлять вышеназванное объектно-ориентированное проектирование прямо в программе (объявление класса есть декларативная часть проекта класса), но уже с учётом синтаксиса этого конкретного языка. Таким образом, программист может задавать характеристики полей данных и определять поведение будущих объектов, руководствуясь лишь особенностями решаемой задачи, не вдаваясь в детали программной реализации методов. Это программирование «сверху вниз» [1].

Все методы класса описываются в разделе определения методов. При этом каждый метод отлаживается независимо от других методов.

Наличие основной программы main, не объявленной ни в одном

классе, весьма удобно и характерно для гибридных языков объектно-ориентированного программирования, к которым относят C++, в отличие, например, от чисто объектно-ориентированных языков, таких, как Java и C# [1]. Гибридные языки соединяют несколько парадигм программирования. Для гибридных языков характерно то, что программа на них может успешно работать без единого класса и объекта. На чисто объектно-ориентированном языке самая простая программа содержит хотя бы один, основной класс.

Для создания консольного приложения на C++ в версии Visual Studio C++ 2019/2022 следует выполнить следующие действия.

1. Загрузить Visual Studio C++ 2019 или Visual Studio C++ 2022.
 2. Выбрать меню «Начало работы» и запустить «Создание проекта».
 3. Выбрав «Консольное приложение» (C++_Widows_Консоль), нажать «Далее».
 4. В окне «Настроить новый проект» присвоить проекту имя, например, ComplexClass, и нажать «Создать».
 5. В открытом окне редактора следует набрать текст программы.
 6. Когда программа набрана, полагается выполнить сборку решения: нажать клавишу F7 или активировать команду меню в окне редактора: «Сборка» -> «Собрать решение».
- После успешной сборки в окне выходных данных будет сообщение *«Сборка: успешно 1, с ошибками: 0, без изменений: 0, пропущено: 0»*. Если есть ошибки, то будет сообщение *«... с ошибками: 1 ...»*. После исправления ошибок – снова нужно провести сборку. Если текст программы между двумя сборками не менялся, будет сообщение *«... без изменений: 1 ...»*.
7. После успешной сборки нужно запустить программу командой CTRL+F5 или командой меню «Отладка» > «Запуск без отладки».

Для сокращения времени программу запускают на выполнение без сборки решения командой CTRL+F5 (или «Отладка» > «Запуск без отладки»). Тогда сборка решения будет также предварительно проведена (по умолчанию). После выполнения программы на экране в консольном окне (белыми буквами на чёрном фоне) появятся результаты работы программы.

В работе с окнами Visual Studio C++ 2019 и Visual Studio C++ 2022 можно делать следующее:

1. В окне «Обозреватель решений» добавлять файлы и управлять ими.
2. В окне «Вид» > “Team Explorer” подключаться к системе управления версиями.
3. В окне «Выходные данные» смотреть выходные сообщения, например, данные сборки.
4. Для создания кода новых файлов использовать меню «Проект» > «Добавить новый элемент»
5. Для добавления в текущий проект кодов уже созданных файлов выбрать пункт «Проект» > «Добавить существующий элемент».
6. Для открытия ранее созданного, но закрытого проекта выбрать пункт меню «Файл» > «Открыть» > «Проект» и загрузить имеющийся sln-файл.

При работе с потоками `cout`, `cin` и `endl`, находящимися в пространстве имён `std`, удобно присоединить к программе пространство имён `std`, расположив после `#include` и перед `main` строку `using namespace std`;

Чтобы при смене на клавиатуре латиницы на кириллицу информация вводилась с клавиатуры по-русски и выводилась на консоль на русском, в Visual Studio C++ 2019 добавляют команды библиотеки `<windows.h>`:

```
SetConsoleCP (1251); // для ввода
```

```
SetConsoleOutputCP (1251); // для вывода
```

В Visual Studio C++ 2022 для этого следует добавить команду из библиотеки `<locale.h>` для вывода на русском:

```
setlocale (LC_ALL, "RUSSIAN");
```

оставив при вводе команду из библиотеки `<windows.h>`:

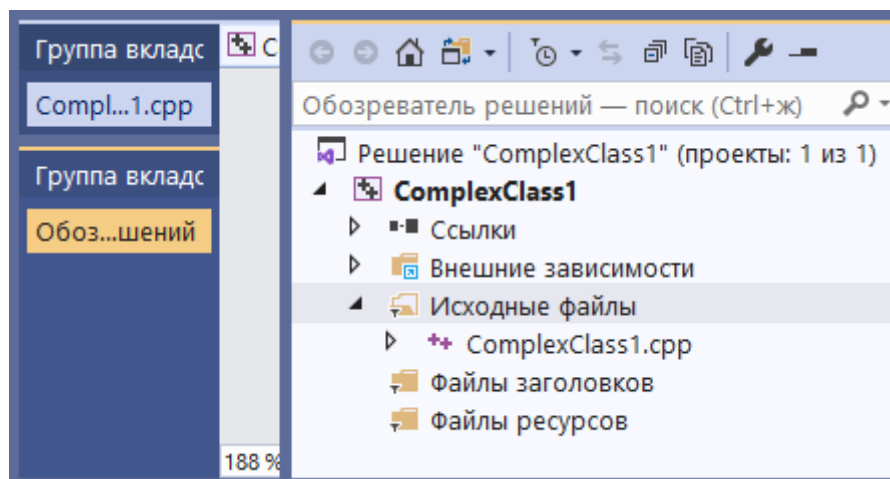
```
SetConsoleCP (1251);
```

По умолчанию консольное окно имеет чёрный фон, а на нём белые буквы. Если в среде Visual Studio C++ нужно сменить фон на белый, а символы сделать чёрными, то нужно использовать `system("color F0");` (библиотека `<windows.h>`). F0 – шестнадцатеричный однобайтовый код цветов фона и символов. Иные сочетания цветов фона и символов можно получить, сменив код F0 на другой.

Исходный текст объектно-ориентированной программы может быть полностью помещён в один единственный файл, как в примере 1.2. Однако часто удобно структурировать проект, поместив вышеназванные отдельные части программы в разные файлы. Тогда в один файл (с расширением `.cpp`) помещают основную программу `main`, в другом файле (с расширением `.h`) размещают раздел объявления классов (заголовочный header-файл), третий же файл (с расширением `.cpp`) содержит определения методов классов.

Чтобы создать такой проект, нужно открыть «Обозреватель решений» напрямую или через меню (Рис. 1.). Например, в приведённом выше случае можно дать проекту новое имя `ComplexClass1` и, открыв файл `ComplexClass1.cpp`, поместить туда раздел основной программы `main`.

Рис.1.2. Окно «Обозреватель решений»



Текст файла `ComplexClass1.cpp` приведён в примере 1.3.

Пример 1.3. Файл ComplexClass1.cpp с основной программой

```
#include <iostream>
#include <windows.h>
#include "Complex.h"    // подключение заголовочного файла

using namespace std;

int main()
{
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    system("color f0");
    Complex C1(3, 4), C2(5, 12), C3;
    cout<< "Результаты работы программы: " <<endl;
    C1.PrintComplex();
    C1.PrintTrig();
    C2.PrintComplex();
    C2.PrintTrig();
    cout << endl << "C3:" << endl;
    C3 = sum(C1, C2); // вызов самостоятельной функции sum
    C3.PrintComplex();
    C3.PrintTrig();
    C3 = C1.Sum(C2); // вызов функции, члена класса (метода Sum)
    C3.PrintComplex();
    C3.PrintTrig();
    system("Pause");
    return 0;
}
```

После этого в окне «Обозреватель решений» следует активировать «Файл заголовков». Откроется окно меню, в котором следует выбрать «Добавить класс» и задать имя класса: Complex. Нажав на «ОК», программист автоматически создаст два файла: заголовочный файл Complex.h в папке «Файлы заголовков» (см. Рис.1.) и исходный файл Complex.cpp. В заголовочном файле Complex.h, служебная строка #pragma once формируется средой автоматически, как и строки class Complex{ }:

```
#pragma once
class Complex
{
};
```

В файл Complex.h следует поместить раздел объявлений классов Complex.h, как в примере 1.4. Здесь же может быть объявлена функция сложения двух комплексных чисел Complex sum(Complex, Complex);

Пример 1.4. Заголовочный файл Complex.h с объявлением класса.

```
#pragma once
class Complex
{
    private:      // закрытая секция класса
        double Re; // действительная часть
        double Im; // мнимая часть
    public:      // открытая секция класса
        Complex(double, double); // конструктор класса
        double GetRe(); // возврат значения поля Re
        double GetIm();
        void PutRe(double); // изменение значения поля Re
        void PutIm(double);
        void PrintComplex(); // печать числа в алгебраической форме
        void PrintTrig(); // печать числа в тригонометрической форме
        Complex Sum(Complex); // сложение объекта с аргументом
    private:      // закрытая секция класса
        double Amp(); // вычисление модуля комплексного числа
        double Fi(); // вычисление фазы комплексного числа
};

Complex sum(Complex, Complex); // функция для сложения двух чисел
```

Определения методов класса Complex и определение функции sum должны быть записаны в файле Complex.cpp, который автоматически создаётся при создании файла Complex.h и размещается либо в папке «Исходные файлы» (вместе с файлом ComplexClass1.cpp), либо в папке «Файлы ресурсов».

Важно заметить, что в каждом из исходных файлов подключаются библиотеки, содержащие именно те функции, что вызываются в этих файлах. Например, в файле ComplexClass1.cpp подключается библиотека <windows.h>, содержащая функции SetConsoleCP, SetConsoleOutputCP и system, а в файле Comple.cpp подключается библиотека <math.h>, содержащая функции sqrt и atan2. Так как в обоих этих файлах используются потоки ввода-вывода cout и cin, то в обоих файлах подключается библиотека <iostream>. Чтобы не помещать перед cout и cin указание на пространство имён std::, в каждом из этих двух исходных файлов помещается команда using namespace std;

Отдельно следует рассмотреть то, как подключается к обоим этим исходным файлам заголовочный файл Complex.h. Это делается с помощью команды #include "Complex.h". В отличие от имён библиотек Visual Studio C++, которые записываются в угловых скобках, имя этого заголовочного файла, созданного программистом, записывается в кавычках.

После команды меню «Собрать решение» все файлы этого проекта компилируются и компонуются в единый исполнительный файл, а по команде меню «Запуск без отладки» происходит выполнение программы.

Пример 1.5. Исходный файл Complex.cpp с определением методов класса

```
#include <iostream>
#include <math.h>
#include "Complex.h"

using namespace std;

Complex::Complex(double Re, double Im)
{
    this->Re = Re;
    this->Im = Im;
}

double Complex::GetRe() { return Re; }

double Complex::GetIm() { return Im; }

void Complex::PutRe(double Re) { this->Re = Re; }

void Complex::PutIm(double Im) { this->Im = Im; }

void Complex::PrintComplex() {cout<<" Re = "<<Re<<" Im = "<<Im<<endl;}

double Complex::Amp() { return sqrt(Re * Re + Im * Im); }

double Complex::Fi() { return atan2(Im, Re); }

void Complex::PrintTrig() {cout<<" A = "<<Amp()<<" Fi = "<<Fi()<<endl;}

Complex Complex::Sum(Complex B)
{
    Complex C(0, 0);
    C.PutRe(Re + B.GetRe());
    C.PutIm(Im + B.GetIm());
    return C;
}

Complex sum(Complex A, Complex B)
{
    Complex C(0, 0);
    C.PutRe(A.GetRe() + B.GetRe());
    C.PutIm(A.GetIm() + B.GetIm());
    return C;
}
```

Рис. 1.3. Текст с результатами работы многофайлового проекта

Результаты работы программы:

Re = 3 Im = 4

A = 5 Fi = 0.927295

Re = 5 Im = 12

A = 13 Fi = 1.17601

C3:

Re = 8 Im = 16

A = 17.8885 Fi = 1.10715

Re = 8 Im = 16

A = 17.8885 Fi = 1.10715

Одинаковые результаты работы однофайлового и многофайлового проектов показывают идентичность обеих программ.

Как видно из текста файла Complex.cpp, функции-члены (методы) и обычные функции определяются по-разному. Перед методами класса обязательно следует указывать имя того класса, членами которого они являются, с двойным двоеточием, например, Complex::

Complex **Complex::**Sum(Complex B){...}

В то же время функция sum определяется обычным образом:

Complex sum(Complex A, Complex B){...}

Также важно обратить внимание на то, что и метод Sum, и функция sum имеют объекты в качестве как входных аргументов, так и в качестве возвращаемых значений. Среды Visual Studio C++ 2019 и Visual Studio C++ 2022 для успешной работы таких функций требуют в теле этих функций задать возвращаемое значение по умолчанию:

Complex C(0, 0);

При этом сложение комплексных чисел $C3 = C1 + C2$ с использованием функции-члена Sum выглядит как $C3 = C1.Sum(C2)$; а с использованием самостоятельной функции sum выглядит как $C3 = sum(C1, C2)$;

Важно заметить, что инкапсуляция, как было сказано выше, представляет собой соединение структур данных и методов их обработки, а в языке C++ структуры данных, как известно, создаются с помощью синтаксической конструкции struct. В средах Visual Studio C++ 2019/2022 простая объектно-ориентированная программа может быть написана без использования ключевого слова class, вместо которого может быть использовано ключевое слова struct. При этом внутри структурированного типа могут быть объявлены не только поля (переменные-члены), но и методы (функции-члены). По умолчанию в структуре struct все компоненты (и поля, и методы) открыты, в отличие от закрытых (по умолчанию) компонентов (и полей, и методов) внутри класса class. В примере 1.6 показан текст объектно-ориентированной программы, использующей struct. Программа аналогична программе в примере 1.2, но значительно упрощена.

Пример 1.6. Объектно-ориентированная программа, использующая struct

```
#include <iostream>
#include <windows.h>
using namespace std;

struct Complex // структура Compl
{
    double Re;    // поля структуры
    double Im;    //
    Complex Sum(Complex); // функция-член
};

Complex sum(Complex, Complex); // самостоятельная функция

int main()
{
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    system("color f0");
    Complex A, B, C;
    B.Re = 5; B.Im = 4;
    cout << "Результаты работы программы со struct:" << endl;
    cout << " B: " << " Re = " << B.Re << " Im = " << B.Im << endl;
    C.Re = 2; C.Im = -1;
    cout << " C: " << " Re = " << C.Re << " Im = " << C.Im << endl;
    A = sum(B,C); // A = B + C
    cout << " A: " << " Re = " << A.Re << " Im = " << A.Im << endl;
    A = B.Sum(C); // A = B + C
    cout << " A: " << " Re = " << A.Re << " Im = " << A.Im << endl;
}

Complex Complex::Sum(Complex Y)
{
    Complex Z;
    Z.Re = Re + Y.Re;
    Z.Im = Im + Y.Im;
    return Z;
}

Complex sum(Complex X, Complex Y)
{
    Complex Z;
    Z.Re = X.Re + Y.Re;
    Z.Im = X.Im + Y.Im;
    return Z;
}
```

Рис. 1.4. Текст с результатами работы проекта со struct вместо class

Результаты работы программы со struct:

B: Re = 5 Im = 4

C: Re = 2 Im = -1

A: Re = 7 Im = 3

A: Re = 7 Im = 3

В объектно-ориентированной программе со struct вместо class можно использовать те же конструкции, что и рассмотренные выше конструкции в программе с ключевым словом class: открытые и закрытые секции public и private, конструкторы, методы акцессоры. Однако такая опция характерна только для последних версий Visual Studio C++. Использование стандартной конструкции class предпочтительно с позиции переносимости кода и общих принципов объектно-ориентированного программирования на C++.

Весьма распространённой ошибкой при работе с объектами является мнение, что на основе одного класса можно создать только один объект. По этой причине в задании к данной работе требуется создать от трёх объектов, имеющих разное поведение. Предлагается в качестве объектов создавать движущиеся геометрические фигуры, наглядно иллюстрирующие решение задачи. У этих геометрических объектов следует одни поля и методы использовать для управления центрами объектов, а другие – для управления размерами этих фигур относительно их центров. Сложные движения нужно создавать как комбинацию простых движений. Поведение каждого из созданных объектов должно быть задано отдельным конкретным методом, отличным от иных. Одни методы могут задавать простые движения как относительно геометрического центра, так и движения самого центра. Другие методы моделируют сложные движения, комбинируя простые согласованным вызовом методов, моделирующих простые движения.

В примере 1.7 показан проект класса Wheel, а в примере 1.8 приведена программа, решающая задачу:

«Разработав класс “Wheel”, создать объекты, изображающие:

- а) “колесо”, которое вращается*
- б) “колесо”, которое скользит*
- в) “колесо”, которое катится».*

Эти методы моделируют лишь элементарные составляющие движений, а движения помещены в основную программу.

В программе использован указатель на компоненты класса **this->** в конструкторе и в акцессорах, чтобы преодолеть неоднозначность работы компилятора *<поле> X= <аргумент>X* в ситуации, когда имя входного аргумента совпадает с именем поля данных. Так, например, вместо метода

```
PutX (int NewX) { X = NewX; }
```

будет метод

```
PutX (int X) { this -> X = X; }.
```

Пример 1.7. Проект класса Wheel

Имя класса: Wheel

Поля данных (закрытые):

X, Y — целые экранные координаты центра колеса;

R — целый радиус колеса;

fi — вещественный угол поворота;

c - цвет колеса типа size_t для Visual Studio C++ 2019/2022.

Методы (открытые):

Конструктор Wheel, входные параметры — целочисленные начальные значения полей данных R, X, Y и вещественное начальное значение поля данных Fi, а также цвет объекта c, имеющий трёхбайтовый тип size_t.

Метод-акцессор: GetX возвращает целое значение поля X;

Иные методы:

Show — рисует объект цветом c как обод с двумя осями (окружность радиуса R с центром в точке (X, Y) и два проходящих через центр объекта отрезка, пересекающихся под прямым углом). Координаты концов отрезков, соединяющих точки 1 и 3, а также 2 и 4. вычисляют по формулам:

$$\begin{aligned}X_1 &= X + R \cdot \sin(\varphi), & Y_1 &= Y - R \cdot \cos(\varphi), \\X_2 &= X + R \cdot \sin\left(\varphi + \frac{\pi}{2}\right), & Y_2 &= Y - R \cdot \cos\left(\varphi + \frac{\pi}{2}\right), \\X_3 &= X + R \cdot \sin(\varphi + \pi), & Y_3 &= Y - R \cdot \cos(\varphi + \pi), \\X_4 &= X + R \cdot \sin\left(\varphi + \frac{3\pi}{2}\right), & Y_4 &= Y - R \cdot \cos\left(\varphi + \frac{3\pi}{2}\right),\end{aligned}$$

где переменным ΔX , φ , $\Delta\varphi$, X_1 , Y_1 , X_2 , Y_2 , X_3 , Y_3 , X_4 , Y_4 .

соответствуют идентификаторы dx, fi, dfi, X1, Y1, X2, Y2, X3, Y3, X4, Y4.

Hide — прячет колесо, чертя его цветом фона.

Slide(целый dx) — двигает колесо по оси X на dx пикселей (прячет методом Hide, увеличивает X на DX и рисует методом Show в новом месте);

Turn(вещественный dfi) — поворачивает колесо на угол dfi (прячет методом Hide, увеличивает fi на dfi и рисует методом Show в новом месте);

Roll(вещественный dfi) — перекачивает колесо по оси X (двигает центр колеса на dx методом Slide, поворачивает вокруг его центра на dfi методом Turn). Сдвиг dx для создания зримого эффекта перекачивания синхронизирован с углом dfi и рассчитывается как $dx = dfi * R$.

Программист сам вправе решать, делать ему однофайловый проект или многофайловый. И, хотя в больших программных продуктах всегда используют многофайловые проекты, в настоящем учебном пособии часто будут представлены в качестве примеров именно однофайловые проекты. Расположение текста объектно-ориентированной программы в одном файле позволяет сэкономить печатный текст, что важно в рамках ограниченного объёма учебного пособия. Освоение изложенных ранее и далее по тексту примеров создания многофайловых проектов позволит читателю без труда самому переделать любой однофайловый проект в многофайловый.

Пример 1.8. Графические объекты на Visual Studio C++ 2019/2022

```
#include <windows.h>
#include <math.h>
#include <iostream>

class Wheel
{
private:
    short R;
    short X, Y;
    float fi;
    size_t c;
public:
    Wheel(size_t, short, short, short, float);
    short GetX();
    void Show(); // нарисовать объект
    void Hide(); // спрятать объект
    void Slide(short); // скольжение
    void Turn(float); // вращение
    void Roll(float); // перекачивание
};

void Line(HDC, short, short, short, short); // черчение отрезка

#define Pi 4 * atan(1)

using namespace std;    // подключить пространство имён std

HDC hdc; // объявление контекста устройства

Wheel::Wheel(size_t c, short R, short X, short Y, float fi)
{
    this->c = c;
    this->X = X;
    this->Y = Y;
    this->R = R;
    this->fi = fi;
}

short Wheel::GetX() { return X; }
```

```

void Wheel::Slide(short dX)
{
    Hide();
    X += dX;
    Show();
}

void Wheel::Turn(float dfi)
{
    Hide();
    fi += dfi;
    Show();
}

void Wheel::Roll(float dfi)
{
    Turn(dfi);
    short dx = R * dfi;
    Slide(dx);
}

void Wheel::Show()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, c);
    SelectObject(hdc, Pen); //сделаем перо активным
    short X1, X2, X3, X4, Y1, Y2, Y3, Y4; // координаты концов спиц
    short Ri; // утолщённый радиус колеса
    float RiS, RiC; // рабочие переменные, экономящие вычисления
    for (char i = -1; i < 2; i++) // цикл утолщения линий
    {
        Ri = R + i;
        RiS = Ri * sin(fi);
        RiC = Ri * cos(fi);
        X1 = X + RiS;
        Y1 = Y - RiC;
        X2 = X - RiS;
        Y2 = Y + RiC;
        X3 = X - RiC;
        Y3 = Y - RiS;
        X4 = X + RiC;
        Y4 = Y + RiS;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
        Line(hdc, X1, Y1, X2, Y2);
        Line(hdc, X3, Y3, X4, Y4);
    }
    DeleteObject(Pen);
}

```

```

void Wheel::Hide()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, RGB(241, 241, 241));
    SelectObject(hdc, Pen); //сделаем перо активным
    short X1, X2, X3, X4, Y1, Y2, Y3, Y4; // координаты концов спиц
    short Ri; // утолщённый радиус колеса
    float RiS, RiC; // рабочие переменные, экономящие вычисления
    for (char i = -1; i < 2; i++)
    {
        Ri = R + i;
        RiS = Ri * sin(fi);
        RiC = Ri * cos(fi);
        X1 = X + RiS;
        Y1 = Y - RiC;
        X2 = X - RiS;
        Y2 = Y + RiC;
        X3 = X - RiC;
        Y3 = Y - RiS;
        X4 = X + RiC;
        Y4 = Y + RiS;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
        Line(hdc, X1, Y1, X2, Y2);
        Line(hdc, X3, Y3, X4, Y4);
    }
    DeleteObject(Pen);
}

void Line(HDC hdc, short x1, short y1, short x2, short y2)
{
    MoveToEx(hdc, x1, y1, NULL);
    LineTo(hdc, x2, y2);
}

int main()
{
    system("color f0");// цвет окна консоли - неяркий белый (светло-серый)
    HWND hwnd = GetConsoleWindow(); // создать графическое окно
    if (hwnd != NULL)
    {
        hdc = GetWindowDC(hwnd); // открыть графическое окно
        short R = 100; // радиус колёс
        /* создание трёх объектов класса Wheel */
        Wheel V1(RGB(0, 0, 255), R, 350, 200, 0);
        Wheel V2(RGB(0, 255, 0), R, 800, 450, Pi * 0.125);
        Wheel V3(RGB(255, 0, 0), R, 350, 700, Pi * 0.25);
    }
}

```

```

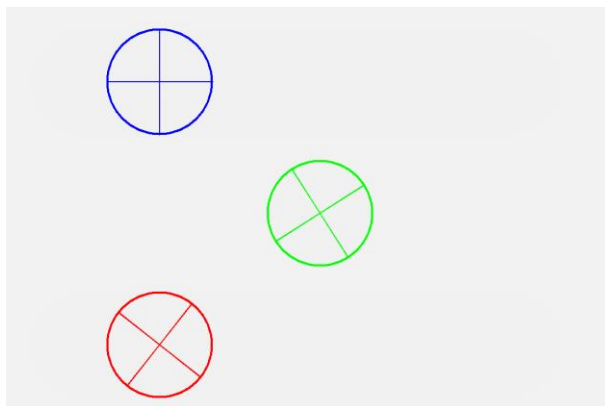
float dfi = 0.05, dfi2 = dfi, dfi3 = dfi;
short dx = R * dfi, dx1 = dx;
if (hdc != 0)
{
    getchar();
    V1.Show();
    V2.Show();
    V3.Show();
    getchar();
    while (1) // цикл движения объектов
    {
        V1.Slide(dx1);
        if (V1.GetX() < 241 || V1.GetX() > 1159) dx1 = -dx1;
        V2.Turn(dfi2);
        V3.Roll(dfi3);
        if (V3.GetX() < 241 || V3.GetX() > 1159) dfi3 = -dfi3;
        Sleep(12); // задержка во времени
        if (GetKeyState(VK_SHIFT) < 0) break; //прерван нажатым shift
    }
    getchar();
    CloseWindow(hwnd); // Закрыть графическое окно
}
}
return 0;
}

```

Результат работы программы с графическими объектами представлен на Рис. 1.5. Это движение трёх объектов, где верхний скользит, средний вращается, а нижний катится, совершая синхронно скольжение и вращение. GetKeyState(VK_SHIFT) прерывает движение объектов нажатием SHIFT.

Для создания графических объектов в примере 1.8 использованы графические функции из библиотеки windows.h в среде Visual Studio C++ 2019/2022. Это функции CreatePen, SelectObject, DeleteObject, Arc, LineTo, MoveToEx, GetConsoleWindow, GetWindowDC, CloseWindow. Встроенные типы из windows.h позволяют: HWND - создать графическое окно, HPEN - сделать перо активным, HDC – задать контекст графического устройства. Подробное разъяснение приёмов работы с графикой в среде Visual Studio C++ 2019/2022 с использованием библиотеки windows.h выходит за рамки настоящего учебного пособия и предполагается к изучению в рамках других дисциплин, либо к самостоятельному изучению. Ведь графические объекты не являются конечной целью при изучении объектно-ориентированного программирования, а только служат хорошей иллюстрацией различных объектно-ориентированных приёмов. Однако каждый, выполняя задания к работе, вправе использовать представленные примеры как начальные заготовки, которые несложно переделать под свою задачу.

Рис. 1.5. Результаты работы проекта с графическими объектами



Задание для лабораторной работы № 1

1. Класс «плоский кружок». Объекты:
 - а) вращается вокруг горизонтальной оси,
 - б) движется без вращения, как подброшенный вверх,
 - в) движется как подброшенный с вращением вокруг горизонтали.
2. Класс «плоский кружок». Объекты:
 - а) вращается вокруг вертикальной оси,
 - б) скользит по горизонтали без вращения,
 - в) скользящей по горизонтали с вращением вокруг вертикали.
3. Класс "окружность". Объекты:
 - а) расширяется (сужается) с неподвижным центром,
 - б) скользит вдоль прямой, не меняя радиуса,
 - в) соединяет оба движения, приближаясь и удаляясь в перспективе.
4. Класс "эллипс". Объекты выполняют:
 - а) расширение-сжатие горизонтально (до окружности) без перемещения,
 - б) скольжение по горизонтали с постоянными размерами,
 - в) "переползание" (синхронное расширение-сжатие и скольжение).
5. Класс "треугольник". Объекты выполняют:
 - а) вращение вокруг его неподвижного центра,
 - б) скольжение по горизонтали,
 - в) "перекатывание" треугольника, синхронно вращая и скользя.
6. Класс «упругий эллипс». Объекты:
 - а) синусоидально меняется вдоль вертикали при неподвижном центре,
 - б) синусоидально колеблется по вертикали при постоянных размерах,
 - в) пружинит, неподвижный снизу, синхронно двигаясь и деформируясь.
7. Класс "равнобедренный треугольник". Объекты выполняют:
 - а) вращение вокруг его геометрического центра,
 - б) обращение вокруг внешнего центра, без вращения,
 - в) обращение, когда одна вершина всегда смотрит на центр обращения.
8. Класс "ромб". Объекты изображают:
 - а) растяжение-сжатие горизонтальной диагонали без перемещения,
 - б) равномерное перемещение по горизонтали,
 - в) "переползание", синхронизируя перемещение и растяжение-сжатие.

9. Класс "прямоугольник". Объекты изображают:
- а) горизонтальное растяжение-сжатие длины стороны без перемещения,
 - б) равномерное перемещение по горизонтали,
 - в) "переползание" при синхронном перемещении и растяжении-сжатии.
10. Класс «египетский овал»: две полуокружности с отрезками. Объекты:
- а) растягиваются-сжимаются изменением длин отрезков без сдвигов,
 - б) равномерно перемещаются по горизонтали,
 - в) "переползают", синхронно перемещая, растягивая и сжимая.
11. Класс "треугольник". Объекты изображают:
- а) растяжение-сжатие по горизонтали изменением основания без сдвига,
 - б) равномерное перемещение по горизонтали,
 - в) "переползание", соединяя сдвиги синхронно с растяжением-сжатием.
12. Класс "ромб". Объекты изображают:
- а) вращение вокруг своего центра,
 - б) обращение вертикально стоящего объекта около внешнего центра,
 - в) обращение, когда вершина направлена на внешний центр вращения.
13. Класс "квадрат". Объекты изображают:
- а) вращение вокруг своего центра,
 - б) обращение вертикально стоящего объекта около внешнего центра,
 - в) обращение, когда сторона направлена на внешний центр вращения.
14. Класс "упругий прямоугольник". Объекты изображают:
- а) синусоидальное растяжение-сжатие по вертикали без перемещения,
 - б) синусоидальное перемещение по вертикали,
 - в) движение, неподвижное снизу, синхронно сдвигая и деформируя.
15. Класс "эллипс". Объекты изображают:
- а) пропорциональное растяжение-сжатие по обеим осям,
 - б) перемещение вдоль наклонной прямой,
 - в) движение в перспективе, сложив перемещение и растяжение-сжатие.
16. Класс "параллелограмм" (не прямоугольный). Объекты изображают:
- а) растяжение-сжатие по горизонтали,
 - б) скольжение по горизонтали,
 - в) "переползание" при синхронных скольжении и растяжении-сжатии.
17. Класс "равнобедренная трапеция". Объекты изображают:
- а) растяжение-сжатие по горизонтали,
 - б) скольжение по горизонтали,
 - в) "переползание" при синхронных скольжении и растяжении-сжатии.
18. Класс "прямоугольная трапеция". Объекты изображают:
- а) растяжение-сжатие по горизонтали,
 - б) скольжение по горизонтали,
 - в) "переползание" при синхронных скольжении и растяжении-сжатии.

Методические указания

Изучить основные понятия в [1-4]. Методы, задающие движения, должны быть простыми и иметь мало входных аргументов. Всё, что задаёт размеры и форму объектов, а не перемещения, разместить в методе Show.

РАБОТА № 2. НАСЛЕДОВАНИЕ КЛАССОВ НА C++ ДЛЯ VISUAL STUDIO 2019/2022

Цель работы – изучение приёмов объектно-ориентированного программирования с наследования классов в средах Visual Studio C++ 2019 и Visual Studio C++ 2022. Освоение понятий класс-наследник и базовый класс, а также понятия защищённые компоненты класса. Особенности взаимодействия конструкторов в иерархии наследования классов на языке C++. Работа с наследуемыми компонентами. Преимущества и недостатки использования наследования.

Общие сведения

В объектно-ориентированной программе может быть создан не один, а несколько классов. Экземпляры каждого из этих классов (объекты) могут быть успешно использованы в основной программе. Однако очень часто встречается ситуация, когда многие переменные-члены и функции-члены в различных классах синтаксически и семантически (по форме и по смыслу) совпадают. Для ускорения разработки классов и для сокращения объёма программного кода имеется механизм продуктивной связи между классами, называемый **наследование** классов.

Несмотря на то, что термин *наследование* в отношениях между людьми предполагает, что человек назначает того человека, кто будет его наследником, в механизме наследования классов, наоборот, в *классе-наследнике* назначается класс, который будет объявлен *базовым*. Класс-наследник получает от базового класса все открытые (public) и защищённые (protected) поля и методы класса (все открытые и защищённые переменные-члены и функции-члены). Иными словами, класс-наследник получает от базового класса все компоненты, не являющиеся закрытыми (не private). При этом наследуемым полям и методам в классе-наследнике может быть дан, если понадобится, другой содержательный смысл.

Понятие *защищённый* (protected) в тексте этого учебного пособия пока ещё не встречалось и должно быть разъяснено. **Защищённые** компоненты базового класса (поля и методы) объявляются в защищённой секции, начинающейся со слова **protected**: Эти защищённые компоненты доступны только методам своего класса и классу-наследнику. Вне базового класса такие переменные-члены и функции-члены недоступны. Если класс не входит ни в какую иерархию наследования, то нет смысла создавать защищённые (protected) секции, так как такие секции по смыслу будут совпадать с закрытыми (private) секциями.

За счёт того, что унаследованные от базового класса методы и поля не нужно заново создавать в классе-наследнике, можно получить следующие основные преимущества, характерные для наследования:

- а) сэкономить объём исходного программного кода,
- б) ускорить разработку объектно-ориентированных программ.

Важно заметить, что, формально, все унаследованные переменные-члены и функции-члены базового класса доступны в классе-наследнике. Но, как правило, не все наследуемые от базового класса компоненты нужны классу-наследнику. Следовательно, при наследовании от базового класса могут быть получены и лишние, не нужные компоненты. При компиляции для работы с полями и методами каждого объекта выделяется оперативная память. В результате наследование, приводя к экономии исходного кода, одновременно может приводить к росту используемой оперативной памяти [1, 9]. Запретить использование не нужных, но унаследованных в классе-потомке методов, возможно с помощью механизма переопределения методов (override). Однако сэкономить используемую оперативную память при этом не удаётся [9].

Конструктор базового класса не является конструктором класса-наследника. Но к открытым и защищённым конструкторам базового класса можно обратиться в конструкторах и иных методах класса-наследника. Вызов конструктора базового класса из конструктора класса-наследника для инициализации наследуемых полей в объектно-ориентированных программах не обязателен. Но в C++ не допускается прямая инициализация унаследованных полей в конструкторе класса-наследника, а требуется обязательный вызов конструктора базового класса. Чтобы преодолеть это, в C++, используют приёмы, рассмотренные ниже. Особенности работы с наследованием показаны ниже в объектно-ориентированных программах. В примере 2.1 даны проекты не связанных между собой классов Point и Circle, а в примере 2.2 приведена иерархия наследования классов Point и Circle, где класс Circle наследует базовому классу Point.

Пример 2.1. Проекты независимых друг от друга классов Point и Circle

Имя класса: Point - «точка»

Поля данных (закрытые):

целые X и Y – экранные координаты «точки»;

трёхбайтовый C типа size_t – код цвета «точки».

Методы (открытые):

а) Конструктор Point задаёт начальные значения полей X, Y и C;

б) Методы-акцессоры

GetX – возвращает значение поля X;

GetY – возвращает значение поля Y;

GetC – возвращает значение поля C;

PutX (новое X) – задаёт значение поля X;

PutY (новое Y) – задаёт значение поля Y;

PutC (новый C) – задаёт значение поля C.

в) Прочие методы

Show – чертит точку на экране;

Hide – скрыть точку, изображая цветом фона;

MoveTo(целые NX, NY) – сдвиг точки в позицию с координатами NX и NY (прячут, задают новые координаты и снова рисуют).

Имя класса: Circle – «окружность»

Поля данных (закрытые): целые X, Y - координаты центра;

целый R – величина радиуса окружности.

трёхбайтовый C типа size_t – код цвета «точки»

Методы (открытые):

а) Конструктор Circle задаёт начальные значения полей X, Y, C и R.

б) Методы-акцессоры

GetX – возвращает значение поля X;

GetY – возвращает значение поля Y;

GetC – возвращает значение поля C;

GetR – возвращает значение поля R;

PutX (новое X) – задаёт значение поля X;

PutY (новое Y) – задаёт значение поля Y;

PutC (новый C) – задаёт значение поля C;

PutR (новый R) – задаёт значение поля R.

в) Прочие методы

Draw – чертит окружность на экране;

Clean – стирает окружность, рисуя цветом фона;

Move (целые NX, NY) – двигает окружность в позицию с координатами

NX и NY (объект стирают, задают координаты центра и снова чертят);

Expand (целое DR) – расширяет окружность на величину DR (окружность стирают, задают новый R и снова рисуют).

Видно, что многие поля и методы классах Point повторятся в классе Circle.

Пример 2.2. Проекты класса Point и класса Circle, наследника класса Point

Имя класса: Point - «точка»

Поля данных (защищённые):

целые X и Y – экранные координаты «точки»;

трёхбайтовый C типа size_t – код цвета «точки».

Методы (открытые):

а) Конструктор Point задаёт начальные значения полей X, Y и C;

б) Методы-акцессоры

GetX – возвращает значение поля X;

GetY – возвращает значение поля Y;

GetC – возвращает значение поля C;

PutX (новое X) – задаёт значение поля X;

PutY (новое Y) – задаёт значение поля Y;

PutC (новый C) – задаёт значение поля C.

в) Прочие методы

Show – чертит точку на экране;

Hide – скрыть точку, изображая цветом фона;

MoveTo(целые NX, NY) – сдвиг точки в позицию с координатами NX и NY (прямчут, задают новые координаты и снова рисуют).

Имя класса: Circle – «окружность», наследует базовому классу Point

Поля данных (защищённые): целый R – радиуса окружности;

Поля (переменные-члены) X, Y и C наследуются из класса Point.

Методы (открытые):

а) Конструктор Circle: задаёт начальные значения X, Y, C и R.

б) Методы – акцессоры

GetR – возвращает значение поля R;

PutR (новый R) – задаёт значение поля R.

Методы (функции-члены) GetX, GetY, GetC,

а также PutX, Put и PutC, наследуются из класса Point.

в) Прочие методы

Draw – чертит окружность на экране;

Clean – стирает окружность, рисуя цветом фона;

Move (целые NX, NY) – двигает окружность в позицию с координатами

NX и NY (объект стирают, задают координаты центра и снова чертят);

Expand (целое DR) – расширяет окружность на величину DR (окружность стирают, задают новый R и снова рисуют).

Методы Show, Hide и MoveTo класса Circle также наследуются из класса Point, но не нужны в классе Circle из-за имеющихся в классе Circle методов Draw, Clear и Move, выполняющих действия, схожие с Show, Hide и MoveTo, но только с объектами класса Circle.

Сравнивая между собой проекты классов Point и Circle из примера 2.1, можно увидеть, что поля X, Y, C, а также методы GetX, GetY, GetC, PutX, PutY и PutC из класса Point полностью повторяются в классе Circle, прежде всего, содержательно. По этой же причине повторяются и их имена. Наличие в одной программе двух классов, частично схожих между собой, но никак не связанных друг с другом, заставляет создать продуктивную связь, объявив класс Circle наследником класса Point на том основании, что все поля (переменные-члены), имеющиеся в классе Point, также имеются в наличии и в классе Circle. Таким образом, объявив класс Circle наследником класса Point, оказывается возможным не объявлять в классе Circle поля, задающие экранные координаты X, Y и цвет C, и связанные с этими полями методы-акцессоры (функции-члены) GetX, GetY, GetC, PutX, PutY и PutC, получив их от класса Point. При этом поля X, Y и C в классе Point следует объявить не закрытыми (private), как в не связанных друг с другом классах, а защищёнными (protected). Выше говорилось, что в классе-наследнике устанавливается, какой класс будет базовым. В базовом классе наследник не назначается. Единственное, что может указывать на возможное наличие наследников у класса, это объявление в базовом классе защищённых (protected) компонент вместо закрытых (private). В позднейших версиях современных объектно-ориентированных языков существует возможность запретить наследование отдельных компонент, объявив их final или sealed.

Пример 2.3. Классы без наследования на Visual Studio C++ 2019/2022

```
#include <windows.h>
#include <iostream>

class Point
{
private:
    short X;
    short Y;
    size_t C;
public:
    Point(short, short, size_t);
    short GetX();
    short GetY();
    size_t GetC();
    void PutX(short);
    void PutY(short);
    void PutC(size_t);
    void Show(); // нарисовать объект
    void Hide(); // спрятать объект
    void MoveTo(short, short);
};

class Circle
{
private:
    short X;
    short Y;
    size_t C;
    short R;
public:
    Circle(short, short, short, size_t);
    short GetX();
    short GetY();
    size_t GetC();
    short GetR();
    void PutX(short);
    void PutY(short);
    void PutC(size_t);
    void PutR(short);
    void Draw(); // нарисовать объект
    void Clear(); // спрятать объект
    void Move(short, short);
};
```

```
HDC hdc; // объявление контекста устройства
```

```
Point::Point(short X, short Y, size_t C)
```

```
{  
    this->X = X;  
    this->Y = Y;  
    this->C = C;  
}
```

```
short Point::GetX() { return X; }
```

```
short Point::GetY() { return Y; }
```

```
size_t Point::GetC() { return C; }
```

```
void Point::PutX(short NX) { X = NX; }
```

```
void Point::PutY(short NY) { Y = NY; }
```

```
void Point::PutC(size_t NC) { C = NC; }
```

```
void Point::Show()
```

```
{  
    HPEN Pen = CreatePen(PS_SOLID, 2, C);  
    SelectObject(hdc, Pen); //сделаем перо активным  
    short R = 2, Ri;  
    for (char i = -1; i < 2; i++) // цикл утолщения точки  
    {  
        Ri = R * i;  
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);  
    }  
    DeleteObject(Pen);  
}
```

```
void Point::Hide()
```

```
{  
    HPEN Pen = CreatePen(PS_SOLID, 2, RGB(241, 241, 241));  
    SelectObject(hdc, Pen);  
    short R = 2, Ri;  
    for (char i = -1; i < 2; i++)  
    {  
        Ri = R * i;  
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);  
    }  
    DeleteObject(Pen);  
}
```

```

void Point::MoveTo(short NX, short NY)
{
    Hide();
    PutX(NX);
    PutY(NY);
    Show();
}

Circle::Circle(short X, short Y, short R, size_t C)
{
    this->X = X;
    this->Y = Y;
    this->R = R;
    this->C = C;
}

short Circle::GetX() { return X; }

short Circle::GetY() { return Y; }

short Circle::GetR() { return R; }

size_t Circle::GetC() { return C; }

void Circle::PutX(short NX) { X = NX; }

void Circle::PutY(short NY) { Y = NY; }

void Circle::PutR(short NR) { R = NR; }

void Circle::PutC(size_t NC) { C = NC; }

void Circle::Draw()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, C);
    SelectObject(hdc, Pen); //сделаем перо активным
    short Ri;
    for (char i = -1; i < 2; i++) // цикл утолщения точки
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

```



```

void Circle::Clear()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, RGB(241, 241, 241));
    SelectObject(hdc, Pen); //сделаем перо активным
    short Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Circle::Move(short NX, short NY)
{
    Clear();
    PutX(NX);
    PutY(NY);
    Draw();
}

int main()
{
    system("color f0"); // цвет окна консоли неяркий белый (светло-серый)
    HWND hwnd = GetConsoleWindow(); // создать графическое окно
    if (hwnd != NULL)
    {
        hdc = GetWindowDC(hwnd); // открыть графическое окно
        /* создание двух объектов класса Point и двух объектов класса Circle */
        Point P1(250, 200, RGB(0, 0, 255));
        Point P2(800, 160, RGB(255, 0, 0));
        Circle C1(250, 500, 50, RGB(0, 255, 255));
        Circle C2(800, 160, 70, RGB(255, 255, 0));
        if (hdc != 0)
        {
            P1.Show(); getchar();
            P2.Show(); getchar();
            C1.Draw(); getchar();
            C2.Draw(); getchar();
            P1.MoveTo(360, 700); getchar();
            P2.MoveTo(230, 820); getchar();
            C1.Move(610, 120); getchar();
            C2.Move(420, 250); getchar();
        }
    }
    CloseWindow(hwnd); // Закрыть графическое окно
    return 0;
}

```

Пример 2.4. Наследование классов на Visual Studio C++ 2019/2022

```
#include <windows.h>
#include <iostream>

class Point
{
protected:
    short X;
    short Y;
    size_t C;
public:
    Point(short, short, size_t);
    short GetX();
    short GetY();
    size_t GetC();
    void PutX(short);
    void PutY(short);
    void PutC(size_t);
    void Show(); // нарисовать объект
    void Hide(); // спрятать объект
    void MoveTo(short, short);
};

class Circle: public Point // класс Circle - наследник базового класса Point
{
protected:
    short R;
public:
    Circle(short, short, short, size_t);
    short GetR();
    void PutR(short);
    void Draw(); // нарисовать объект
    void Clear(); // спрятать объект
    void Move(short, short);
};

HDC hdc; // объявление контекста устройства

Point::Point(short X, short Y, size_t C)
{
    this->X = X;
    this->Y = Y;
    this->C = C;
}
```

```

short Point::GetX() { return X; }
short Point::GetY() { return Y; }
size_t Point::GetC() { return C; }
void Point::PutX(short NX) { X = NX; }
void Point::PutY(short NY) { Y = NY; }
void Point::PutC(size_t NC) { C = NC; }

void Point::Show()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, C);
    SelectObject(hdc, Pen); //сделаем перо активным
    short R = 2, Ri;
    for (char i = -1; i < 2; i++) // цикл утолщения точки
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Point::Hide()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, RGB(241, 241, 241));
    SelectObject(hdc, Pen); //сделаем перо активным
    short R = 2, Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Point::MoveTo(short NX, short NY)
{
    Hide();
    PutX(NX);
    PutY(NY);
    Show();
}

```

```

// Вызов конструктором класса Circle конструктора базового класса Point
Circle::Circle(short X, short Y, short IR, size_t C): Point(X, Y, C) {R = IR;}

short Circle::GetR() { return R; }

```

```

void Circle::PutR(short NR) { R = NR; }

void Circle::Draw()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, GetC());
    SelectObject(hdc, Pen); //сделаем перо активным
    short Ri;
    for (char i = -1; i < 2; i++) // цикл утолщения точки
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Circle::Clear()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, RGB(241, 241, 241));
    SelectObject(hdc, Pen); //сделаем перо активным
    short Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Circle::Move(short NX, short NY)
{
    Clear();
    PutX(NX);
    PutY(NY);
    Draw();
}

int main()
{
    system("color f0");// цвет окна консоли неяркий белый (светло-серый)
    HWND hwnd = GetConsoleWindow(); // создать графическое окно
    if (hwnd != NULL)
    {
        hdc = GetWindowDC(hwnd); // открыть графическое окно
        /* создать объекты классов Point и Circle*/
        Point P1(250, 200, RGB(0, 0, 255));
        Point P2(800, 160, RGB(255, 0, 0));
        Circle C1(250, 500, 50, RGB(0, 255, 255));
        Circle C2(800, 160, 70, RGB(255, 255, 0));
    }
}

```

```

        if (hdc != 0)
        {
            P1.Show(); getchar();
            P2.Show(); getchar();
            C1.Draw(); getchar();
            C2.Draw(); getchar();
            P1.MoveTo(360, 700); getchar();
            P2.MoveTo(230, 820); getchar();
            C1.Move(610, 120); getchar();
            C2.Move(420, 250); getchar();
        }
    }
    CloseWindow(hwnd); // Закрыть графическое окно
    return 0;
}

```

Текст программы из примера 2.4 существенно короче, чем текст программы из примера 2.3 за счёт использования наследования классов. Важно заметить то, как определён конструктор класса Circle:

```
Circle::Circle(short X, short Y, short IR, size_t C): Point (X, Y, C) {R = IR;}
```

Это характерная особенность наследования классов на C++.

Более естественная в программировании конструкция, где один метод вызывается из другого метода :

```

Circle::Circle(short X, short Y, short IR, size_t C)
{
    Point(X, Y, C);
    R = IR;
}

```

в этой программе невозможна. Невозможна и аналогичная программная конструкция:

```

Circle::Circle(short IX, short IY, short IR, size_t IC)
{
    X = IX;
    Y = IY;
    C = IC;
    R = IR;
}

```

Чтобы эта последняя конструкция успешно функционировала, следует в класс Point добавить ещё один конструктор, конструктор «по умолчанию», например, такой:

```
Point() {};
```

Такой приём называется перегрузка методов (overload) и будет более подробно рассмотрен позднее.

Однофайловый проект из примера 2.4 переделан в многофайловый в примере 2.5, также как однофайловый проект из примера 1.2 переделан в многофайловый проект в примерах 1.3-1.5. Исходный файл Inheritance.cpp содержит текст основной программы, в заголовочном файле PointCircle.h представлены объявления классов Point и Circle проекта, а исходный файл PointCircle.cpp содержит определения методов классов Point и Circle.

Пример 2.5. Многофайловый проект из трёх файлов
с наследование классов на Visual Studio C++ 2019/2022

Исходный файл Inheritance.cpp с текстом основной программы:

```
#include <windows.h>
#include <iostream>
#include "PointCircle.h"

HDC hdc; // объявление контекста устройства

int main()
{
    system("color f0"); // цвет окна консоли неяркий белый (светло-серый)
    HWND hwnd = GetConsoleWindow(); // создать графическое окно
    if (hwnd != NULL)
    {
        hdc = GetWindowDC(hwnd); // открыть графическое окно
        /* создание объектов классов Point и Circle */
        Point P1(250, 200, RGB(0, 0, 255));
        Point P2(800, 160, RGB(255, 0, 0));
        Circle C1(250, 500, 50, RGB(0, 255, 0));
        Circle C2(800, 160, 70, RGB(255, 0, 0));

        if (hdc != 0)
        {
            getchar();
            P1.Show(); getchar();
            P2.Show(); getchar();
            C1.Draw(); getchar();
            C2.Draw(); getchar();
            P1.MoveTo(360, 700); getchar();
            P2.MoveTo(230, 820); getchar();
            C1.Move(610, 120); getchar();
            C2.Move(420, 250); getchar();
        }
        CloseWindow(hwnd); // Закрыть графическое окно
    }
    return 0;
}
```

Заголовочный файл PonitCircle.h с объявлениями классов Point и Circle:

```
#pragma once

class Point
{
protected:
    short X;
    short Y;
    size_t C;
public:
    Point(short, short, size_t);
    short GetX();
    short GetY();
    size_t GetC();
    void PutX(short);
    void PutY(short);
    void PutC(size_t);
    void Show(); // нарисовать объект
    void Hide(); // спрятать объект
    void MoveTo(short, short);
};

class Circle : public Point
{
protected:
    short R;
public:
    Circle(short, short, short, size_t);
    short GetR();
    void PutR(short);
    void Draw(); // нарисовать объект
    void Clear(); // спрятать объект
    void Move(short, short);
};
```

Файл PonitCircle.cpp с определениями методов класса Point и Circle:

```
#include <windows.h>

#include "PointCircle.h"

extern HDC hdc; // внешняя переменная объявления контекста устройства
```

```

Point::Point(short X, short Y, size_t C)
{
    this->X = X;
    this->Y = Y;
    this->C = C;
}

short Point::GetX() { return X; }
short Point::GetY() { return Y; }
size_t Point::GetC() { return C; }
void Point::PutX(short NX) { X = NX; }
void Point::PutY(short NY) { Y = NY; }
void Point::PutC(size_t NC) { C = NC; }
void Point::MoveTo(short NX, short NY)
{
    Hide();
    PutX(NX);
    PutY(NY);
    Show();
}

Circle::Circle(short X, short Y, short IR, size_t C) : Point(X, Y, C) { R = IR; }
short Circle::GetR() { return R; }
void Circle::PutR(short NR) { R = NR; }
void Circle::Move(short NX, short NY)
{
    Clear();
    PutX(NX);
    PutY(NY);
    Draw();
}

void Point::Show()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, C);
    SelectObject(hdc, Pen); //сделать перо активным
    short R = 2, Ri;
    for (char i = -1; i < 2; i++) // цикл утолщения точки
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

```



```

void Point::Hide()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, RGB(241, 241, 241));
    SelectObject(hdc, Pen);
    short R = 2, Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Circle::Draw()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, GetC());
    SelectObject(hdc, Pen);
    short Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Circle::Clear()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, RGB(241, 241, 241));
    SelectObject(hdc, Pen);
    short Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

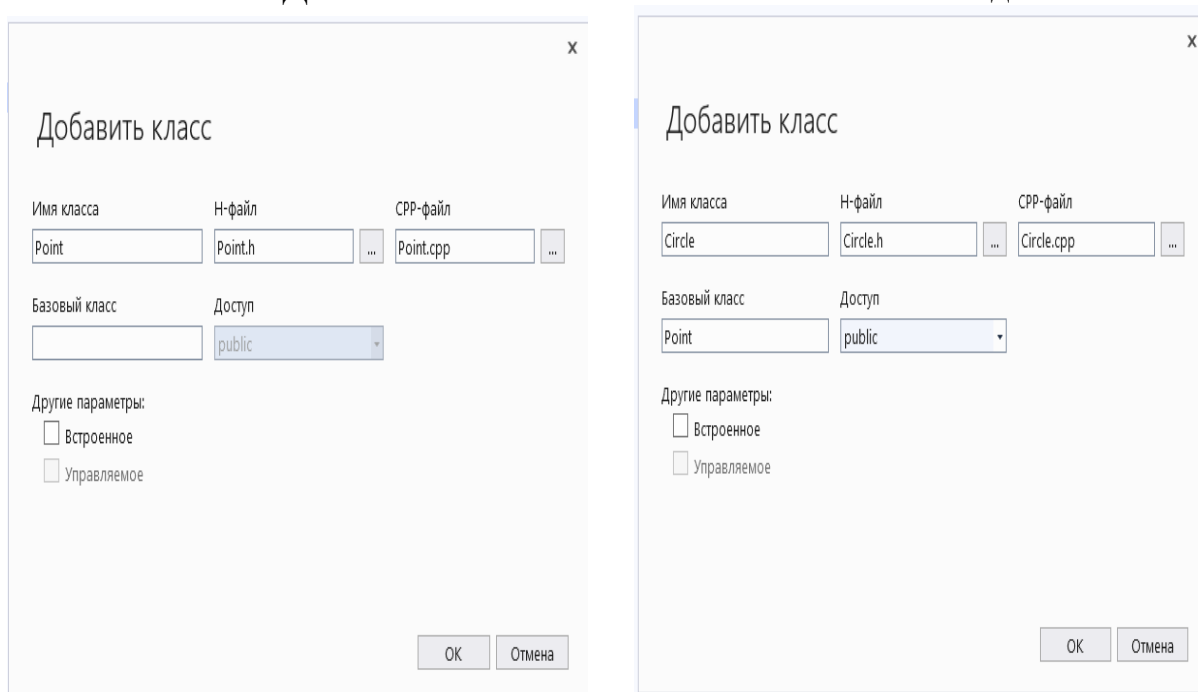
```

Важно заметить, что исходные файлы многофайлового проекта из примера 2.5 содержат не только вызов заголовочного файла PointCircle.h командой #include "PointCircle.h", но, для поддержания работы графических функций из библиотеки windows.h, оба исходных файла содержат глобальную переменную, объявляющую контекст графического устройства: HDC hdc; причём в одном из исходных файлов hdc - внешняя переменная:

extern HDC hdc;

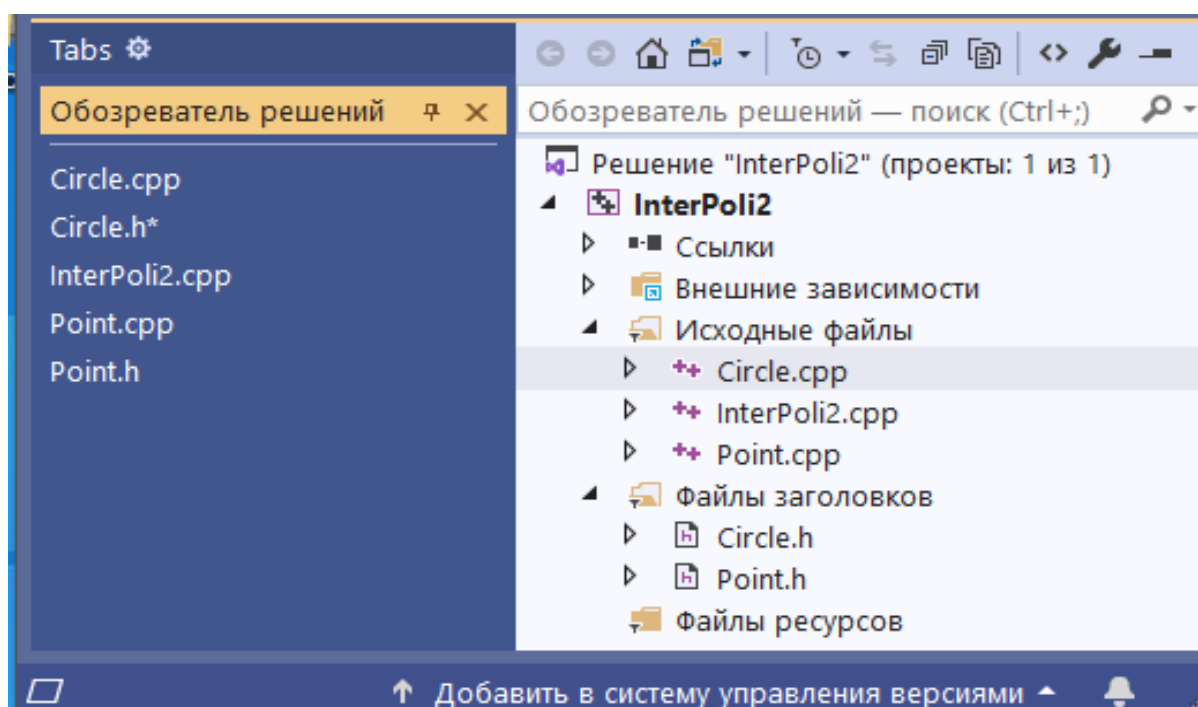
В средах Visual Studio C++ 2019/2022 имеется иной способ, когда для каждого класса создаётся свой заголовочный файл и свой исходный файл: в *обозревателе решений* кнопки: *Файлы заголовков -> Добавить -> Класс*. В появившемся окне «Добавить класс» нужно набрать имя класса: Point. Автоматически появятся имена Point.h и Point.cpp. Нажать ОК (см. Рис. 2.1):

Рис. 2.1. Окно «Добавить класс» класса Point и класса-наследника Circle



Заполнить открытое окно файла Point.h, затем заполнить файл Point.cpp. После этого в «обозревателе решений» добавить класс Circle (см. Рис.2.2): В «обозревателе решений» (Рис. 2.2.) все 5 файлов: 2 h-файла и 3 cpp- файла.

Рис. 2.2. Окно «Обозреватель решений» при создании проекта с наследованием классов



В примере 2.6 представлены все пять файлов многофайлового проекта InterPoli2 с наследованием классов: исходный файл InterPoli2 с основной программой, заголовочный файл Point.h с объявлением класса Point, исходный файл Point.cpp, содержащий определения методов класса Point, заголовочный файл Circle.h с объявлением класса Circle, исходный файл Circle.cpp с определениями методов классов Circle.

Пример 2.6. Многофайловый проект InterPoli2 из пяти файлов
с наследование классов на Visual Studio C++ 2019/2022

Файл InterPoli2:

```
#include <windows.h>
#include <iostream>
#include "Point.h"
#include "Circle.h"

HDC hdc; // объявление контекста устройства

int main()
{
    system("color f0"); // цвет окна консоли неяркий белый (светло-серый)
    HWND hwnd = GetConsoleWindow(); // создать графическое окно
    if (hwnd != NULL)
    {
        hdc = GetWindowDC(hwnd); // открыть графическое окно
        /* создание объектов классов Point и Circle */
        Point P1(250, 200, RGB(0, 0, 255));
        Point P2(800, 160, RGB(255, 0, 0));
        Circle C1(250, 500, 50, RGB(0, 255, 0));
        Circle C2(800, 160, 70, RGB(255, 0, 0));
        if (hdc != 0)
        {
            getchar();
            P1.Show(); getchar();
            P2.Show(); getchar();
            C1.Draw(); getchar();
            C2.Draw(); getchar();
            P1.MoveTo(360, 700); getchar();
            P2.MoveTo(230, 820); getchar();
            C1.Move(610, 120); getchar();
            C2.Move(420, 250); getchar();
        }
        CloseWindow(hwnd); // Закрыть графическое окно
    }
    return 0;
}
```

Файл Point.h:

```
#pragma once

class Point
{
protected:
    short X;
    short Y;
    size_t C;
public:
    Point(short, short, size_t);
    short GetX();
    short GetY();
    size_t GetC();
    void PutX(short);
    void PutY(short);
    void PutC(size_t);
    void Show(); // нарисовать объект
    void Hide(); // спрятать объект
    void MoveTo(short, short);
};
```

Файл Circle.h:

```
#pragma once

#include "Point.h"

class Circle: public Point
{
protected:
    short R;
public:
    Circle(short, short, short, size_t);
    short GetR();
    void PutR(short);
    void Draw(); // нарисовать объект
    void Clear(); // спрятать объект
    void Move(short, short);
};
```

Файл Point.cpp:

```
#include "Point.h"
#include <windows.h>

extern HDC hdc; // внешняя переменная объявления контекста устройства

Point::Point(short X, short Y, size_t C)
{
    this->X = X;
    this->Y = Y;
    this->C = C;
}
short Point::GetX() { return X; }
short Point::GetY() { return Y; }
size_t Point::GetC() { return C; }
void Point::PutX(short NX) { X = NX; }
void Point::PutY(short NY) { Y = NY; }
void Point::PutC(size_t NC) { C = NC; }
void Point::Show()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, C);
    SelectObject(hdc, Pen); //сделать перо активным
    short R = 2, Ri;
    for (char i = -1; i < 2; i++) // цикл утолщения точки
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}
void Point::Hide()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, RGB(241, 241, 241));
    SelectObject(hdc, Pen);
    short R = 2, Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}
void Point::MoveTo(short NX, short NY)
{
    Hide();
    PutX(NX);
    PutY(NY);
    Show();
}
```

Файл Circle.cpp:

```
#include "Circle.h"
#include <windows.h>

extern HDC hdc; // внешняя переменная объявления контекста устройства

Circle::Circle(short X, short Y, short IR, size_t C) : Point(X, Y, C) { R = IR; }

short Circle::GetR() { return R; }

void Circle::PutR(short NR) { R = NR; }

void Circle::Draw()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, GetC());
    SelectObject(hdc, Pen);
    short Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Circle::Clear()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, RGB(241, 241, 241));
    SelectObject(hdc, Pen);
    short Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Circle::Move(short NX, short NY)
{
    Clear();
    PutX(NX);
    PutY(NY);
    Draw();
}
```

Задание для лабораторной работы №2

1. Разработать, в соответствии с вариантом, не связанные друг с другом классы, предусмотрев аксессоры для каждой переменной-члена. Создать функции-члены, моделирующие поступательные движения и вращения объектов, а также меняющие размеры объектов. Это должны быть разные методы в разных классах (Show, Hide и MoveTo у предка, Draw, Clean и Move у потомка в примерах 2.1—2.6).

2. Создать иерархию наследования классов.

3. Создать объекты разработанных классов. Показать все возможные изменения размеров и положения созданных объектов на экране.

4. Обратиться к объектам классов-потомков с помощью унаследованных от классов-предков функций-членов вроде Show и Draw, MoveTo и Move в примерах 2.1—2.6. Убедиться, что результаты таких вызовов функций-членов неправильны.

5. Разработать функции-члены, возвращающие значение геометрических параметров (*длин, площадей и объёмов*) объектов этих классов.

6. Сравнить объёмы программных кодов исходных файлов программы без наследования классов и объёмы исходных файлов всех программ с наследованием классов.

7. Оформить отчёт, как указано в тексте об общем порядке выполнения лабораторных работ.

Варианты иерархий классов

1. точка => отрезок => квадрат => куб
2. точка => отрезок => квадрат => четырёхугольная пирамида
3. точка => отрезок => квадрат => антипризма с квадратами в основании
4. точка => отрезок => параллелограмм => параллелепипед
5. точка => отрезок => ромб => призма с ромбом в основании
6. точка => отрезок => треугольник => тетраэдр
7. точка => отрезок => треугольник => треугольная призма
8. точка => отрезок => окружность => цилиндр
9. точка => отрезок => окружность => сфера
10. точка => отрезок => прямоугольник => прямоугольная пирамида
11. точка => отрезок => прямоугольник => прямоугольный параллелепипед
12. точка => отрезок => эллипс => эллипсоид вращения
13. точка => отрезок => эллипс => эллипсоид трёхосный
14. точка => отрезок => окружность => конус
15. точка => отрезок => ромб => ромбическая пирамида
16. точка => отрезок => эллипс => эллиптический цилиндр
17. точка => отрезок => окружность => конус
18. точка => отрезок => параллелограмм => пирамида с параллелограммом в основании

Методические указания

При выполнении работы нужно изучить наследование классов и освоить понятие защищённые (protected) компоненты класса [1–8].

Разрабатывая функции-члены, отвечающие за разные деформации и повороты, нужно создавать лишь такие, что, с точки зрения геометрии, проще реализовать. Единый способ для всех подобных функций-членов, такой: «Для изменения положения, ориентации или формы геометрического объекта, его должно спрятать, изменить положение, ориентацию или размер, и снова изобразить объект».

Вместе с преимуществами и достоинствами механизма наследования, изложенными в [1–8], должно знать о недостатки наследования, о чём говорится в [1, 9].

РАБОТА № 3. ПОЛИМОРФИЗМЫ. ПЕРЕГРУЗКА И ПЕРЕОПРЕДЕЛЕНИЕ МЕТОДОВ

Цель работы – изучение разных видов полиморфизма методов (функций-членов) в объектно-ориентированном программировании, а именно, перегрузки (overload) методов внутри класса и переопределения (override) методов при наследовании классов. Обзор преимуществ, которые даёт переопределение, на примере программ с наследованием. Удобства в программировании, которые предоставляет перегрузка.

Общие сведения

Наследуемые от базового класса методы (наследуемые функции-члены) могут быть изменены в классе-наследнике. Их код может быть переопределён. Термин **переопределение** (override) обозначает широко распространённый на практике приём программирования, который является разновидностью **полиморфизма** (многоформенности) методов. Для того, чтобы переопределить унаследованный метод, необходимо объявить этот метод в классе-наследнике, после чего заново определить его в этом классе. Тогда к объектам класса-наследника можно будет послать сообщения уже с помощью переопределённых методов. Переопределение методов позволяет избежать вызова унаследованных функций-членов базового класса, которые выполняют действия, похожие, но не совпадающие с теми, что выполняют перегруженные функции-члены класса-наследника.

В примере 2.4 можно видеть, что методы Draw, Clear и Move класса Circle выполняют действия, похожие на действия методов Show, Hide и MoveTo, соответственно, унаследованные от класса Point. Но в классе Circle присутствуют методы и методы Draw, Clear и Move, и методы Show, Hide и MoveTo. В силу попарной схожести методов Draw и Show, Clear и Hide, Move и MoveTo, программист при работе с объектами класса Circle может

ошибочно вызвать методы Show, Hide и MoveTo из класса Point, которые не нужны в классе Circle. Наличие в классе Circle этих унаследованных методов ещё и требует дополнительных затрат оперативной памяти, выделяемую для реализации методов Show, Hide и MoveTo в объектах класса Circle.

Чтобы не было ложных вызовов объектами класса Circle методов, унаследованных от класса Point, методы Show, Hide и MoveTo можно переопределить в классе Circle. Тогда в классе Circle не будет методов Draw, Clear и Move. Вместо них теперь будут методы Show, Hide и MoveTo, соответственно. Для этого в классе Circle вместо методов Draw, Clear и Move нужно объявить методы Show, Hide и MoveTo (как в классе Point), но определение методов Show, Hide и MoveTo в классе Circle должно быть таким, как было определение методов Draw, Clear и Move.

В примере 3.1 представлены: исходный файл InterPoli3 с основной программой, вызывающей из объектов класса Circle переопределённые методы Show, Hide и MoveTo, заголовочный файл Circle.h с объявлениями методов Show, Hide и MoveTo, переопределяемых в классе Circle, а также исходный файл Circle.cpp с определениями (переопределениями) методов Show, Hide и MoveTo проекта InterPoli3. Файлы Point.h и Point.cpp не изменились и остаются такими же, как в примере 2.6.

Пример 3.1. Переопределение методов при наследовании классов на Visual Studio C++ 2019/2022

Файл Circle.h:

```
#pragma once
```

```
#include "Point.h"
```

```
class Circle: public Point // класс Circle - наследник базового класса Point
{
protected:
    short R;
public:
    Circle(short, short, short, size_t);
    short GetR();
    void PutR(short);
    void Show(); // переопределённый метод
    void Hide(); // переопределённый метод
    void MoveTo(short, short); // переопределённый метод
};
```

Файл Circle.cpp:

```
#include <windows.h>
```

```
#include "Circle.h"
```

```
extern HDC hdc; // внешняя переменная объявления контекста устройства
```

```
    // Вызов конструктором класса Circle конструктора базового класса Point  
Circle::Circle(short X, short Y, short IR, size_t C): Point(X, Y, C) {R = IR;}
```

```
short Circle::GetR() { return R; }
```

```
void Circle::PutR(short NR) { R = NR; }
```

```
void Circle::Show() // переопределение метода Show
```

```
{
```

```
    HPEN Pen = CreatePen(PS_SOLID, 2, GetC());
```

```
    SelectObject(hdc, Pen); //сделаем перо активным
```

```
    short Ri;
```

```
    for (char i = -1; i < 2; i++) // цикл утолщения точки
```

```
    {
```

```
        Ri = R * i;
```

```
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
```

```
    }
```

```
    DeleteObject(Pen);
```

```
}
```

```
void Circle::Hide() // переопределение метода Hide
```

```
{
```

```
    HPEN Pen = CreatePen(PS_SOLID, 2, RGB(241, 241, 241));
```

```
    SelectObject(hdc, Pen); //сделаем перо активным
```

```
    short Ri;
```

```
    for (char i = -1; i < 2; i++)
```

```
    {
```

```
        Ri = R * i;
```

```
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
```

```
    }
```

```
    DeleteObject(Pen);
```

```
}
```

```
void Circle::MoveTo(short NX, short NY) //переопределение метода MoveTo
```

```
{
```

```
    Hide(); // вызов переопределённого метода Hide
```

```
    PutX(NX);
```

```
    PutY(NY);
```

```
    Show(); // вызов переопределённого метода Show
```

```
}
```

Файл InterPoly3

```
#include <windows.h>
#include <iostream>
#include "Point.h"
#include "Circle.h"

HDC hdc; // объявление контекста устройства

int main()
{
    system("color f0"); // цвет окна консоли неяркий белый (светло-серый)
    HWND hwnd = GetConsoleWindow(); // создать графическое окно
    if (hwnd != NULL)
    {
        hdc = GetWindowDC(hwnd); // открыть графическое окно
        /* создание объектов классов Point и Circle */
        Point P1(250, 200, RGB(0, 0, 255));
        Point P2(800, 160, RGB(255, 0, 0));
        Circle C1(250, 500, 50, RGB(0, 255, 0));
        Circle C2(800, 160, 70, RGB(255, 0, 0));

        if (hdc != 0)
        {
            getchar();
            P1.Show(); getchar();
            P2.Show(); getchar();
            C1.Show(); getchar();
            C2.Show(); getchar();
            P1.MoveTo(360, 700); getchar();
            P2.MoveTo(230, 820); getchar();
            C1.MoveTo(610, 120); getchar();
            C2.MoveTo(420, 250); getchar();
        }
        CloseWindow(hwnd); // Закрыть графическое окно
    }
    return 0;
}
```

В сравнении с проектом из примера 2.2 и в сравнении с программой в примере 2.4 из работы №2, в проекте 3.1 и в программе 3.2 не нужны имена методов Draw, Clean и Move, так как вместо них в программах используются имена вышеназванных переопределённых методов Show, Hide и MoveTo, а тела – ранее используемых методов Draw, Clean и Move. К сожалению, переопределение методов, так же, как и наследование методов, не приводят к уменьшению используемой оперативной памяти компьютера, как это часто может по началу представляться. Но исследования показывают, что наследование классов увеличивает требуемую оперативную память [1, 9].

В рамках объектно-ориентированного подхода часто разрешается создавать несколько методов (функций — членов класса) с одинаковыми именами даже в рамках одного класса. Такие методы обязательно отличаются друг от друга количеством и/или типом входных параметров. Это другая разновидность полиморфизма методов, так называемая **перегрузка** — *overload*. Для перегрузки методов не нужна иерархия наследования, так как все перегружаемые методы находятся в одном классе. В примере 3.2 представлена перегрузка конструкторов, также называемый полиморфизм конструкторов. Эта перегрузка позволяет инициализировать при создании разных объектов одного класса разное количество полей.

Пример 3.2. Полиморфизм конструкторов на Visual Studio C++ 2019/2022
В классе Point могут быть объявлены два конструктора:

```
Point(short, short, size_t);  
Point(); // конструктор по умолчанию
```

Определение этих конструкторов может выглядеть так:

```
Point:: Point(short X, short Y, size_t C)  
{  
    this -> X = X;  
    this -> Y = Y;  
    this -> C = C;  
}  
  
Point:: Point() // конструктор по умолчанию  
{  
    X = 800;  
    Y = 450;  
    Color = RGB (127, 127, 127);  
}
```

В классе Circle также могут быть объявлены два конструктора:

```
Circle(short, short, short, size_t);  
Circle(); // конструктор по умолчанию
```

В свою очередь, определение этих двух конструкторов может выглядеть так:

```
Circle:: Circle(short X, short Y, short IR, size_t C) : Point(X, Y, C) { R = IR; }  
Circle:: Circle() : Point(800, 450, RGB(127, 127, 127)) { R = 100; }
```

или так:

```
Circle:: Circle(short X, short Y, short IR, size_t C)  
{  
    Point(X, Y, C);  
    R = IR;  
}
```

```
Circle::Circle() // конструктор по умолчанию
{
    X = 800;
    Y = 450;
    R = 100;
    C = RGB(128, 128, 128);
}
```

Кроме того, конструктор класса Circle с четырьмя аргументами может выглядеть и так:

```
Circle::Circle(short IX, short IY, short IR, size_t IC)
{
    X = IX;
    Y = IY;
    R = IR;
    C = IC;
}
```

Таким образом, присущее языку C++ требование, чтобы при определении конструктора класса-наследника обязательно вызывался конструктор базового класса через двоеточие после списка входных аргументов, может быть отменено при наличии конструктора по умолчанию у базового класса. Объекты классов Point и Circle могут быть созданы в теле функции main с использованием всех перегруженных конструкторов:

```
int main()
{
    ...
    Point P1(700, 500, RGB(0, 0, 255));
    Point P2;
    ...
    Circle C1(395, 500, 120, RGB(0, 255, 0));
    Circle C2;
    ...
}
```

Указатель this

В примере 3.2 в одном из конструкторов класса Point использован указатель this. Такой указатель означает, что компонент, на который он указывает, есть переменная-член или функция-член класса. Наличие этого указателя позволяет избавиться от неоднозначности в методе. Например:

```
Circle::Circle(short X, short Y, short InitR, size_t C): Point(X,Y,C) {R = InitR;}
можно заменить InitR на R. Тогда будет получен код:
```

```
Circle:: Circle(short X, short Y, short R, size_t C) : Point(X, Y, C) { R = R; }
```

Здесь слева от знака = стоит поле, а справа от = стоит входной аргумент. Совпадение имён поля и аргумента - неоднозначность работы компилятора, имеющее вид:

$$R = R;$$

Чтобы не было этой неоднозначности, удобно использовать указатель `this`:

$$\text{this} \rightarrow R = R;$$

В этом случае `this -> R` слева от знака = есть переменная-член, а `R` справа от знака = есть входной аргумент метода.

Таким образом указатель `this` применяется чаще всего. Но это не исключает применения указателя `this` в других синтаксических конструкциях, где это необходимо. Однако часто встречающееся на практике использование указателя `this` повсеместно в функциях-членах не оправдано, так как влечёт за собой неоправданное увеличение объёма исходного кода программы.

Важно знать, что в версиях C++ для Visual Studio 2019/2022 разрешена не только перегрузка (overload) функций-членов, но также и перегрузка самостоятельных функций. В программе могут быть несколько функций с одинаковым именем, но с разным количеством и/или типом аргументов.

Задание для лабораторной работы №3

1. Создать проекты программ из заданий ко 2-й работе с использованием механизма наследования и с переопределением функций-членов, отвечающие за изображение и стирание объектов, и за движение объектов. Устранить разницу в именах переопределяемых методов, например, различия в именах `Show`, `Hide` и `MoveTo` у базового класса, и в именах `Draw`, `Clean` и `Move` у класса-наследника.

2. Нарисовав графические объекты - экземпляры всех разработанных классов, продемонстрировать удобства, которые даёт переопределение функций-членов для избавления от ложных вызовов из объектов классов-наследников функций-членов, унаследованных от базовых классов.

3. Создать все возможные конструкторы в объектно-ориентированных программах, в которых часть полей или все поля инициализируются по умолчанию. Показать объекты, созданные с использованием полиморфных конструкторов.

Методические указания

При подготовке к выполнению задания к этой работе нужно быть знакомым с механизмом переопределения методов (override) в классах-наследниках из разработанной иерархии, изложенным, например, в [1–8]. Также следует изучить перегрузку (overload) методов в классе, рассмотренную в [1, 6]. Наряду с очевидными достоинствами наследования, рассмотренными, например, в [1–8], нужно учесть и недостатки механизма наследования, о которых говорится, в том числе и в [1, 9]. Важно помнить, существует ненулевая вероятность ложного вызова унаследованных, но совсем не нужных функций-членов.

РАБОТА № 4. ПОЗДНЕЕ СВЯЗЫВАНИЕ. ВИРТУАЛЬНЫЕ ФУНКЦИИ-ЧЛЕНЫ

Цель работы - изучение одной из особенностей механизма позднего связывания - виртуальных методов (виртуальных функций-членов). Освоение приёмов создания и использования виртуальных методов. Овладение приёмами экономии исходного кода объектно-ориентированной программы с помощью виртуальных функций.

Общие сведения

Наличие механизма переопределения функций-членов в классах-наследниках даёт возможность избавиться от ложных вызовов унаследованных, но непереопределённых методов из объектов классов-потомков. Переопределение делает работу с иерархией классов более удобной и позволяет использовать дополнительные возможности наследования методов с активным применением виртуальных функций.

Переопределяемые методы в *связанных иерархией наследования* классах могут быть объявлены как **виртуальные**. Виртуальные функции-члены (виртуальные методы, *virtual*), в отличие от обычных, не виртуальных методов, подключаются к объектно-ориентированной программе не на этапе компиляции программы, а на этапе её выполнения. Это называется **позднее связывание**. Особенностью позднего связывания (в отличие от раннего связывания, происходящего при компиляции) является то, что оказывается возможным, работая с объектами классов-потомков, вызывать переопределённые методы классов-потомков из унаследованных методов классов-предков. Возможность правильно использовать унаследованные методы (использование которых ранее приводило к неправильным результатам), позволяет дополнительно экономить программный код.

Например, в работы №3 переопределённый в классе-наследнике Circle метод MoveTo имеют полностью совпадающий код с методом MoveTo из базового класса Point. Разница есть только в имени класса:

```
void Point::MoveTo(short NX, short NY) // метод MoveTo класса Point
{
    Hide();
    PutX(NX);
    PutY(NY);
    Show();
}

void Circle::MoveTo(short NX, short NY) //метод MoveTo класса Circle
{
    Hide(); // вызов переопределённого метода Hide
    PutX(NX);
    PutY(NY);
    Show(); // вызов переопределённого метода Show
}
```

Но метод MoveTo не может быть унаследован классом Circle из класса Point без переопределения, так как метод MoveTo в классе Circle вызывает методы Show и Hide, также переопределённые в классе Circle. Фактически получается, что код метода MoveTo в разных классах разный, хотя по форме код одинаковый. Преодолеть разрыв между формой и содержанием можно, объявив методы Show и Hide виртуальными (virtual). Тогда метод MoveTo может быть унаследован классом Circle без переопределения. При обращении к объектам класса Circle с сообщениями MoveTo в этом случае будут вызваны методы Show и Hide, переопределённые в классе Circle. Таким образом, необходимости переопределять в классе Circle метод MoveTo приводит к экономии объёма программного кода.

При грамотном программировании такой приём экономии может быть распространён и на метод Hide. Ведь прятать графический объект на экране означает рисовать объект цветом фона. Для этого в теле метода Hide нужно сохранить текущий цвет объекта C в рабочей переменной TempColor, затем присвоить объекту светло-серый цвет фона RGB(241, 241, 241), а после вызвать метод Show, что приведёт к рисованию объекта цветом фона, то есть, к стиранию объекта на экране. В завершении метода Hide полно текущего цвета C должно быть присвоено сохранённое значение TempColor, что необходимо для того, чтобы не изменить цвета других графических объектов в программе. Тогда метод Hide не нужно будет переопределять в классе Circle, достаточно будет унаследовать его из класса Point. В итоге виртуальным достаточно объявить только метод Show, переопределённый в классе Circle. А методы Hide и MoveTo достаточно создать один раз в классе Point, с тем чтобы затем наследовать эти методы во всей иерархии.

В целом для графических объектов такая экономия может оказаться весьма весомой, так как иерархия наследования классов может быть очень длинной, а для всех объектов общими являются методы Hide – «нарисовать цветом фона», MoveTo – «спрятать, изменить координаты и нарисовать», а также метод *«повернуть»* - «спрятать, изменить угол поворота, нарисовать», *«деформировать»* - «спрятать, изменить размер, нарисовать», и похожие, по сути, методы. Вместо того, чтобы в каждом классе прописывать свои собственные методы для работы с экземплярами этого класса, достаточно в каждом классе создать виртуальный метод Show, а все остальные методы для работы с графическими объектами создать в верху иерархии, чтобы классы-наследники вызывали эти методы.

Правильное применение виртуальных функций-членов (виртуальных методов) просто проверить. Следует выполнить программу с виртуальными методами, затем снять виртуальность и снова выполнить. В первом случае программа работает правильно, а во втором – должна работать неправильно. Иные результаты покажут, что виртуальность неверно использована.

Особенность языка C++ это то, что, после однократного объявления виртуальным метода в вершине иерархии, все переопределённые методы с таким именем и сигнатурой станут виртуальными. Пример 4.1 содержит многофайловый проект с экономией кода за счёт виртуальных функций.

Пример 4.1. Использование виртуальных функций при наследовании классов с переопределение методов на Visual Studio C++ 2019/2022

Файл InterPoli4:

```
#include <windows.h>
#include <iostream>
#include "Point.h"
#include "Circle.h"

HDC hdc; // объявление контекста устройства

int main()
{
    system("color f0");
    HWND hwnd = GetConsoleWindow(); // создать графическое окно
    if (hwnd != NULL)
    {
        hdc = GetWindowDC(hwnd); // открыть графическое окно
        Point P1(250, 200, RGB(0, 0, 255));
        Point P2(800, 160, RGB(255, 0, 0));
        Circle C1(250, 500, 50, RGB(0, 255, 0));
        Circle C2(800, 160, 70, RGB(255, 0, 0));
        if (hdc != 0)
        {
            getchar();
            P1.Show(); getchar();
            P2.Show(); getchar();
            C1.Show(); getchar();
            C2.Show(); getchar();
            P1.Hide(); getchar();
            P2.Hide(); getchar();
            C1.Hide(); getchar();
            C2.Hide(); getchar();
            P1.Show(); getchar();
            P2.Show(); getchar();
            C1.Show(); getchar();
            C2.Show(); getchar();
            P1.MoveTo(360, 700); getchar();
            P2.MoveTo(230, 820); getchar();
            C1.MoveTo(610, 120); getchar();
            C2.MoveTo(420, 250); getchar();
        }
        CloseWindow(hwnd); // Закрыть графическое окно
    }
    return 0;
}
```

Файл Point.h:

#pragma once

class Point

```
{
    protected:
        short X;
        short Y;
        size_t C;
    public:
        Point(short, short, size_t);
        short GetX();
        short GetY();
        size_t GetC();
        void PutX(short);
        void PutY(short);
        void PutC(size_t);
        virtual void Show(); // виртуальная функция-член
        void Hide(); // наследуемая функция-член
        void MoveTo(short, short); // наследуемая функция-член
};
```

Файл Circle.h:

#pragma once

#include "Point.h"

class Circle: public Point

```
{
    protected:
        short R;
    public:
        Circle(short, short, short, size_t);
        short GetR();
        void PutR(short);
        void Show(); // виртуальная функция-член
};
```

Файл Point.cpp:

```
#include "Point.h"
#include <windows.h>

extern HDC hdc; // внешняя переменная объявления контекста устройства

Point::Point(short X, short Y, size_t C)
{
    this->X = X;
    this->Y = Y;
    this->C = C;
}

short Point::GetX() { return X; }
short Point::GetY() { return Y; }
size_t Point::GetC() { return C; }
void Point::PutX(short NX) { X = NX; }
void Point::PutY(short NY) { Y = NY; }
void Point::PutC(size_t NC) { C = NC; }
void Point::Show()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, C);
    SelectObject(hdc, Pen); // сделать перо активным
    short R = 2, Ri;
    for (char i = -1; i < 2; i++) // цикл утолщения точки
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Point::Hide() // метод для стирания любого графического объекта
{
    size_t TempColor = C; // сохранение текущего цвета
    C = RGB(241, 241, 241); // цвет фона неярко белый (светло-серый)
    Show(); // вызов виртуальной функции: рисование цветом фона
    C = TempColor; // возвращение текущему цвету сохранённого цвета
}

void Point::MoveTo(short NX, short NY) // движение любого объекта
{
    Hide(); // вызов наследуемой функции
    PutX(NX);
    PutY(NY);
    Show(); // вызов виртуальной функции
}
```

Файл Circle.cpp:

```
#include "Circle.h"
#include <windows.h>

extern HDC hdc; // внешняя переменная объявления контекста устройства

Circle::Circle(short X, short Y, short IR, size_t C) : Point(X, Y, C) { R = IR; }

short Circle::GetR() { return R; }

void Circle::PutR(short NR) { R = NR; }

void Circle::Show() // виртуальная функция
{
    HPEN Pen = CreatePen(PS_SOLID, 2, GetC());
    SelectObject(hdc, Pen);
    short Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}
```

Видно, что объём исходного кода в примере 4.1 стал гораздо меньше, чем коды объектно-ориентированных программ, решающих такие же задачи в работах №2 и №3.

Задание для лабораторной работы №4

Используя примеры программ с наследованием и с переопределением методов из работы №2 и работы №3, показать применение виртуальных методов для экономии исходного кода программ: вместе с функциями-членами MoveTo и Hide следует создать единые для разных классов методы, отвечающие за развороты, повороты и за различные деформации.

Сравнить объёмы исходных кодов программ из работ №2, №3 и №4:
без наследования;
с наследованием, но без переопределения методов;
с наследованием, с переопределением методов, без виртуальных функций;
с наследованием, с переопределением методов, с виртуальной функцией-членом.

Методическое указание

Особенности использования виртуальных методов рассмотрены в [5, 10].

РАБОТА № 5. АБСТРАКТНЫЕ КЛАССЫ И ПОЛИМОРФНЫЕ ОБЪЕКТЫ. ИНТЕРФЕЙСЫ (interface) НА Visual Studio C++ 2022

Цель работы – освоение приёмов работы с абстрактными классами и интерфейсами. Развитие на новом уровне идеи абстракции типов данных. Создание чисто-виртуальных функций-членов (абстрактных методов) и абстрактных классов. Разработка и применение полиморфных объектов вместе с абстрактными классами с целью экономии исходного кода программ. Создание и использование интерфейсов (interface).

Общие сведения

Идея абстракции данных активно развивается и широко используется в объектно-ориентированном программировании. Само понятие *класс* и связанное с ним понятие *инкапсуляция* есть реализация этой идеи. Развитие же идеи абстракции данных на новом уровне – абстрактные классы.

Абстрактный класс - это класс, который содержит абстрактный метод (хотя бы один). **Абстрактный метод**, который называется в C++ **чисто виртуальная** функция, есть метод, лишь *объявленный* в своём классе, но *не определённый*. Абстрактный метод, объявленный в базовом классе, должен быть определён в классе-наследнике. То есть, абстрактный класс создаётся обязательно для наследования. Абстрактные классы находятся в вершине иерархии наследования и не могут быть в программе изолировано, вне иерархии наследования. Во многих языках объектно-ориентированного программирования существует запрет на создание объектов абстрактных классов. Там же, где такого запрета нет, объект абстрактного класса не может вызывать абстрактный метод.

В примере 5.1 на языке C++ для Visual Studio 2019/2022 приведён многофайловый проект, аналогичный проекту из примера 2.5, в котором продолжено развитие иерархии наследования классов, рассмотренной в работах №2-4. Из класса Point выделены целые поля X и Y, обозначающие экранные координаты точки и задающие позицию этой точки на экране. Создан абстрактный класс Location (позиция), содержащий целые поля X и Y, конструктор, аксессоры и абстрактный метод Show (чисто виртуальную функцию-член). Абстрактный класс Location сделан базовым классом по отношению к классу Point, который стал классом-наследником. Класс Point, в итоге, стал содержать только поле C, задающее цвет точки. В классе Point остались аксессоры для поля C, конструктор, метод Show, определяющий («переопределяющий») унаследованный от Location абстрактный метод, а также методы Hide и MoveTo. Чтобы лучше видеть текст получившейся иерархии наследования Location-Point-Circle, выбран трёхфайловый проект, аналогичный проекту из примера 2.5, где в файле LocPointCircle.h собраны объявления всех классов, в файле LocPointCircle.cpp имеются определения всех функций-членов (методов) этих классов, а основная программа main находится в файле InherPoliLoc.cpp, совпадающим с именем проекта.

Пример 5.1. Абстрактный класс на языке C++ для Visual Studio 2019/2022

Файл LocPointCircle.h:

```
#pragma once

class Location
{
protected:
    short X;
    short Y;
public:
    Location(short, short);
    short GetX();
    short GetY();
    void PutX(short);
    void PutY(short);
    virtual size_t GetC() = 0; // чисто виртуальная функция (абстрактный метод)
    virtual void PutC(size_t) = 0; // чисто виртуальная функция (абстр-й метод)
    virtual void Show() = 0; // чисто виртуальная функция (абстрактный метод)
};

class Point : public Location
{
protected:
    size_t C;
public:
    Point(short, short, size_t);
    size_t GetC(); // объявление переопределяемого абстрактного метода
    void PutC(size_t); // объявление переопределяемого абстр-го метода
    void Show(); // объявление переопределяемого абстрактного метода
    void Hide();
    void MoveTo(short, short);
};

class Circle : public Point
{
protected:
    short R;
public:
    Circle(short, short, short, size_t);
    short GetR();
    void PutR(short);
    void Show(); // нарисовать объект
};
```

Файл LocPointCircle.cpp:

```
#include <windows.h>
#include "LocPointCircle.h"

extern HDC hdc; // объявление контекста устройства

Location::Location(short X, short Y)
{
    this->X = X;
    this->Y = Y;
}

short Location::GetX() { return X; }
short Location::GetY() { return Y; }

void Location::PutX(short NX) { X = NX; }
void Location::PutY(short NY) { Y = NY; }

Point::Point(short X, short Y, size_t IC) : Location(X, Y) { C = IC; }

size_t Point::GetC() { return C; }

void Point::PutC(size_t NC) { C = NC; }

void Point::Show() // определение абстрактного метода
{
    HPEN Pen = CreatePen(PS_SOLID, 2, C);
    SelectObject(hdc, Pen); //сделать перо активным
    short R = 2, Ri;
    for (char i = -1; i < 2; i++) // цикл утолщения точки
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

void Point::Hide()
{
    size_t TempColor = C;
    C = RGB(241, 241, 241);
    Show();
    C = TempColor;
}

void Point::MoveTo(short NX, short NY)
{
    Hide();
    PutX(NX);
    PutY(NY);
    Show();
}
```

```

Circle::Circle(short X, short Y, short IR, size_t C) : Point(X, Y, C) { R = IR; }

short Circle::GetR() { return R; }

void Circle::PutR(short NR) { R = NR; }

void Circle::Show()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, GetC());
    SelectObject(hdc, Pen);
    short Ri;
    for (char i = -1; i < 2; i++)
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}

```

Файл InherPoliLoc.cpp:

```

#include <windows.h>
#include <iostream>
#include "LocPointCircle.h"
HDC hdc;
int main()
{
    system("color f0");
    HWND hwnd = GetConsoleWindow();
    if (hwnd != NULL)
    {
        hdc = GetWindowDC(hwnd);
        Point P1(250, 200, RGB(0, 0, 255));
        Point P2(800, 160, RGB(255, 0, 0));
        Circle C1(250, 500, 50, RGB(0, 255, 0));
        Circle C2(800, 160, 70, RGB(255, 0, 0));
        if (hdc != 0)
        {
            P1.Show(); getchar();
            P2.Show(); getchar();
            C1.Show(); getchar();
            C2.Show(); getchar();
            P1.MoveTo(360, 700); getchar();
            P2.MoveTo(230, 820); getchar();
            C1.MoveTo(610, 120); getchar();
            C2.MoveTo(420, 250); getchar();
        }
        CloseWindow(hwnd); // Закрыть графическое окно
    }
    return 0;
}

```


В примере 5.1 намеренно все объявления классов собраны в одном заголовочном файле. Так легче и удобнее изучать всю иерархию в целом. Но программист имеет право создавать и более детальный проект, когда каждое объявление класса будет помещено в отдельный файл, и когда все определения методов каждого класса будут собраны вместе в одном исходном файле-срр, соответствующем этому классу, как в примере 2.6 и в примере 3.1.

Показанное создание и использование абстрактного класса возможно в большинстве версий языка C++. Однако в Visual Studio C++ 2019/2022 имеется возможность ещё большей абстракции данных, чем абстрактные классы. Это так называемые интерфейсы - interface.

Интерфейс обозначают ключевым словом **interface** в языке C++ для Visual Studio 2019/2022. В отличие от абстрактного класса (в котором может быть достаточно одного абстрактного метода), интерфейс содержит только объявления (сигнатуры) всех открытых методов. В интерфейсе, в отличие от класса, нет полей данных, нет секций (public, private, protected), так как все объявленные методы по умолчанию открыты. Интерфейс создан для наследования. Все объявленные в интерфейсе открытые методы должны быть определены в классе-наследнике. Например, в файле LocPointCircle.h из примера 5.1 перед классом Location может быть объявлен интерфейс с именем ILocation, которому будет наследовать класс Point:

```
interface ILocation // интерфейс
{
    short GetX();
    short GetY();
    void PutX(short);
    void PutY(short);
};

class Location: public ILocation // класс Location наследует ILocation
{ protected:
    short X;
    short Y;
public:
    Location(short, short);
    short GetX();
    short GetY();
    void PutX(short);
    void PutY(short);
    virtual void Show() = 0; //чисто виртуальная функция (абстрактный метод)
};
```

Это фрагмент иерархии наследования ILocation-Location-Point-Circle. Такая иерархия развивает иерархию Location-Point-Circle. Видно, что здесь интерфейс ILocation содержит объявления (или сигнатуры) всех (исключая конструктор) открытых функций-членов (открытых методов), объявленных в классе Location, который наследует интерфейсу ILocation. Имя интерфейса

выбрано таким для наглядности, но может быть любым, как и у классов. Ограничений на объявления открытых методов в интерфейсе не существует. Вместо приведённого кода интерфейс ILocation может содержать такой код:

```
interface ILocation
{
    short GetX();
    short GetY();
    size_t GetC();
    void PutX(short);
    void PutY(short);
    void PutC(size_t);
    void Show();
    void Hide();
    void MoveTo(short, short);
};
```

Этот код соответствует открытым методам, объявленным во всех классах иерархии Location-Point-Circle, за исключением конструкторов.

Иерархия наследования может не иметь абстрактного класса, но иметь интерфейс. Это будет иерархия IPoint-Point-Circle с интерфейсом IPoint:

```
interface IPoint
{
    short GetX();
    short GetY();
    size_t GetC();
    void PutX(short);
    void PutY(short);
    void PutC(size_t);
    void Show();
    void Hide();
    void MoveTo(short, short);
};
```

```
class Point : public IPoint
{ protected: short X;
    short Y;
    size_t C;
public: Point(short, short, size_t);
    short GetX();
    short GetY();
    size_t GetC();
    void PutX(short);
    void PutY(short);
    void PutC(size_t);
    void Show(); // определение абстрактного метода
    void Hide();
    void MoveTo(short, short);
};
```

Легко видеть, что, в этом случае, текст кода интерфейса IPoint почти совпадает с текстом кода объявления класса Point, за исключением всех полей данных и метода-конструктора, а также за исключением объявлений защищённой и открытой секций.

Интерфейсы interface, разрешённые к применению в C++ для Visual Studio 2019/2022, аналогичны интерфейсам interface в языках Java и C#. В языке C++ такие интерфейсы появились недавно. Ранее интерфейсы в C++ моделировались с помощью абстрактных классов. В учебниках и Интернете распространён такой способ моделирования интерфейсов, когда вместо реального интерфейса объявляется абстрактный класс без полей данных с открытой секцией, состоящей только из одних чисто виртуальных функций (одних абстрактных методов):

```
class ILocation //моделирование интерфейса абстрактным классом
{ public:
    virtual short GetX() = 0;
    virtual short GetY() = 0;
    virtual size_t GetC() = 0;
    virtual void PutX(short) = 0;
    virtual void PutY(short) = 0;
    virtual void PutC(size_t) = 0;
    virtual void Show() = 0;
    virtual void Hide() = 0;
    virtual void MoveTo(short, short) = 0;
};

class Location: public ILocation // класс Location наследует классу ILocation
{ protected:
    short X;
    short Y;
public:
    Location(short, short);
    short GetX();
    short GetY();
    void PutX(short);
    void PutY(short);
};
```

и так далее... Однако видно, что код абстрактного класса, моделирующего интерфейс, более сложный и громоздкий, чем аналогичный код интерфейса.

Интерфейсы в таких объектно-ориентированных языках как Java и C#, где нет разделения на объявление класса и определение методов класса, удобны при проектировании класса в тексте программы, так как методы в Java и в C# определяются на месте прямо в тексте класса, а интерфейсы дают возможность программисту объявить открытый метод, определив его уже в классе-наследнике интерфейса. Но в C++ это не нужно из-за возможности в языке разделять объявление класса и определение методов этого класса.

Экономия исходного кода с использованием указателя на абстрактный тип данных

Абстрактный тип данных позволяет экономить объём кода исходной программы почти так же, как при использовании виртуальной функции-члена из примера 4.1. Вместо наследуемых методов Hide и MoveTo можно создать самостоятельные функции Clear и Move (не методы) с указателем на абстрактный тип как с типом дополнительного аргумента каждой такой функции. Тогда для стирания объекта с экрана нужно вызывать одну и ту же функцию Hide, а для перемещения объекта по экрану следует вызывать также одну и ту же функцию Move, меняя указатели на эти геометрические объекты при подстановке аргумента-указателя. Этот приём можно видеть в примере 5.2.

Пример 5.2. Функции с указателем на абстрактный класс в качестве аргумента на языке C++ для Visual Studio 2019/2022

Взяв за основу пример 5.1, в файле LocPointCircle.h добавить вместо методов Hide и MoveTo объявления функций Clear и Move:

```
void Clear(Location *); // объявление функции Clear
void Move(short, short, Location *); // объявление функции Move
```

а в файле LocPointCircle.cpp добавить вместо определений методов Hide и MoveTo определения функций Clear и Move:

```
void Clear(Location * pL)    // определение функции Clear
{
    size_t TempColor = pL->GetC();
    pL->PutC(RGB(241, 241, 241));
    pL->Show();
    pL->PutC(TempColor);
};

void Move(short X, short Y, Location *pL) // определение функции Move
{
    Clear(pL);
    pL->PutX(X);
    pL->PutY(Y);
    pL->Show();
}
```

Тогда в файле InherPoliLoc.cpp в функцию main после создания объектов:

```
Point P1(250, 200, RGB(0, 0, 255));
Point P2(800, 160, RGB(255, 0, 0));
Circle C1(250, 500, 50, RGB(0, 255, 0));
Circle C2(800, 160, 70, RGB(255, 0, 0));
```

вместо вызовов методов Hide и MoveTo поставить вызовы функций Clear и Move, имеющих полиморфные аргументы:

...

```
Move (150, 250, &P1); getchar();           // полиморфный аргумент у Move
Move (200, 200, &P2); getchar();           // полиморфный аргумент у Move
Move(400, 300, &C1); getchar();           // полиморфный аргумент у Move
Move(300, 400, &C2); getchar();           // полиморфный аргумент у Move
```

...

Видно, что в одной и той же позиции аргумента у функции `Move` и в каждом вызове стоит новый адрес (указатель) нового объекта, тип которого представляет собой указатель на класс, входящий в иерархию классов, начинающуюся с абстрактного класса `Location`. Такой аргумент у функции `Move`, как и у функции `Clear`, следует называть и считать *полиморфным*.

Полиморфный аргумент функции в C++ это такой аргумент, тип которого может меняться во время работы программы. Если нет динамического выделения памяти (нет позднего связывания), то на этапе компиляции будет выделена оперативная память под каждый заданный тип. В языке C++ полиморфный аргумент у функции создаётся с помощью указателя на базовый абстрактный класс в вершине иерархии наследования. Тогда вместо указателя на абстрактный класс при вызове функции в этот аргумент будет подставлено значение указателя на объект одного из классов-наследников базового абстрактного класса.

Вместо подстановки в качестве аргументов функции `Move` указателей на различные объекты из созданной иерархии можно подставлять указатель на полиморфный объект.

Полиморфизм является (вместе с инкапсуляцией и наследованием) одной из основ объектно-ориентированного программирования. Выше, в работе №3, были рассмотрены полиморфизмы методов: переопределение и перегрузка. При изучении объектно-ориентированного программирования кроме полиморфизма методов рассматривают полиморфизм объектов.

Полиморфный объект меняет свой тип во время работы программы. Если в программе есть иерархия наследования, то полиморфным объектом может быть экземпляр произвольного класса из иерархии классов. Полиморфный объект имеет одно и то же имя независимо от типа. Как видно в примере 5.2, для создания такого полиморфизма используется полиморфный указатель. Ему присваиваются в основной программе `main` адреса (указатели) на объекты классов из заданной иерархии наследования. В основной программе `main` процедура `Move` вызывается с одним и тем же полиморфным указателем `pL`, только указывающим на разные объекты. То есть полиморфный объект реализуется в C++ с помощью полиморфного указателя. Продолжая пример 5.2, следует в теле основной функции `main` создать указатель `pL` на абстрактный класс `Location`:

```

Location *pL;      // объявление указателя на абстрактный объект

pL = &P1;          // полиморфный указатель
Move(250, 150, pL); getchar();

pL = &P2;          // полиморфный указатель
Move(300, 300, pL); getchar();

pL = &C1;          // полиморфный указатель
Move(300, 400, pL); getchar();

pL = &C2;          // полиморфный указатель
Move(400, 300, pL); getchar();

```

Видно, что в данном случае при всех вызовах функции Move имя аргумента pL не меняется. Однако при этом указателю pL каждый раз присваивается адрес (указатель) нового объекта, с новым типом. Таким образом, указатель pL оказывается полиморфным указателем.

Задание для работы №5

Добавить в иерархию наследования классов абстрактный класс. Создать программы, используя результаты работ №2-4.

Изучить возможности работы с интерфейсами (interface) в объектно-ориентированной программе на C++ для Visual Studio 2019/2022.

Использовать абстрактные классы и запрограммировать функции с дополнительными аргументами, представляющими собой указатель на абстрактный класс вместо наследуемых методов MoveTo и Hide, имеющих общий и единый для разных классов код.

Показать то, как можно использовать полиморфизм объектов для экономии объёма исходного кода программ. Сравнивая с экономией кода во 2-й и 4-й работах, показать иную, чем при использовании виртуальных методов, возможность экономии исходного кода программы за счёт того, что самостоятельные функции стирания и движения объектов являются общими для объектов всех классов из рассмотренной иерархии.

Методические указания

Прикладная сторона применения абстрактных классов подробно не исследована в литературе по программированию. Абстрактные классы и методы (чисто виртуальные функции) рассмотрены, например, в книгах [1, 3-5]. Использование полиморфных указателей на объекты классов из одной иерархии показано в [3]. Термин *чисто виртуальная функция* в C++ даёт, зачастую, ложное убеждение, что абстрактные методы есть разновидность виртуальных функций. Следует чётко различать методы, созданные для наследования и определения в классах-наследниках (абстрактные) и методы, подключаемые при выполнении программы (виртуальные).

Интерфейсы (interface) нужно изучить на сайте компании Microsoft.

РАБОТА № 6. МЕТОД КОМПОЗИЦИИ. ПОДОБЬЕКТЫ

Цель работы – изучение композиции - особого приёма объектно-ориентированного программирования, устанавливающего новые связи между классами, отличные от связей, создающихся при наследовании. Освоение новых синтаксических конструкций при использовании композиции на C++ для Visual Studio 2019|2022.

Общие сведения

В объектно-ориентированных программах возможно использование в качестве полей данных одного класса объекты любого другого класса. Этот приём в литературе называется **композиция** (см. [1, 10]). Он позволяет создавать между классами новую связь, не такую, как при наследовании.

Композиция (а также *агрегация*, [8]) позволяет создавать программу, более близкую к реальной задаче, чем к синтаксису конкретного языка объектно-ориентированного программирования.

Применение композиции требует знаний особенностей создания классов, когда типы полей данных, а также типы аргументов конструкторов существенно отличаются от встроенных типов данных языка C++. Также при таком подходе могут быть совершенно особыми как типы аргументов, так и тексты других методов.

Прежде всего, при разработке и программировании классов на C++ следует создать, наряду с традиционно необходимыми конструкторами, конструкторы «по умолчанию», как в примерах, показанных далее. В этих примерах показаны классы, имеющие в качестве полей данных экземпляры других классов (*подобъекты*, как их называют в [6]). Важно учесть объекты классов, использующиеся в композиции. В качестве входных аргументов конструкторов классов, имеющих поля-подобъекты, соответствующие объекты должны быть заранее созданы в программе.

Чтобы более полно освоить композицию, задания к этой работе разделены на две части:

В одной нужно использовать композицию по принципу «матрёшки». Тогда в композиционной иерархии объекты каждого класса становятся полями следующего за данным классом по иерархии класса. Объект каждого уровня в иерархии может быть изображён массивом геометрическим объектов более низкого уровня. Эта иерархия композиции объектов с классами типа «точка–отрезок–решётчатый_квадрат–решётчатая_пирамида».

Другая часть задания предполагает, что объекты любого из нижних уровней могут быть полями классов более высокого уровня. Например, композиционная иерархия «точка–отрезок–треугольник–тетраэдр», где фигура задаётся точками-вершинами и может иметь либо стороны, либо рёбра и грани. При таком подходе в одном объекте могут быть сразу несколько экземпляров разных классов такой иерархии на равных правах.

Пример 6.1. Проект Compozition с композицией классов и объектов
на Visual Studio C++ 2019/2022

Файл PointLine.h:

/ объявления классов Point и Line */*

#pragma once

class Point

{

private:

short X;

short Y;

size_t C;

public:

Point(short, short, size_t);

Point(); *// обязательный конструктор "по умолчанию"*

void PutX(short);

void PutY(short);

void PutC(size_t);

short GetX();

short GetY();

size_t GetC();

void Show();

void Hide();

void MoveTo(short, short);

};

class Line

{

public:

Point P1; *// поля данных класса Line -*

Point P2; *// экземпляры (объекты) класса Point*

size_t C;

Line(**Point**, **Point**, size_t);

void Show();

void Hide();

void Move(short, short);

};

void line(HDC, **Point**, **Point**); *// функция рисования отрезка*

Файл PointLine.cpp:

```
#include <windows.h>
#include <iostream>
#include "PointLine.h"

extern HDC hdc; // объявление контекста устройства

/* определение объявленных методов классов Point и Line */
Point::Point(short X, short Y, size_t C)
{
    this->X = X;
    this->Y = Y;
    this->C = C;
}

Point::Point()
{
    X = 0;
    Y = 0;
}

void Point::PutX(short X) { this->X = X; }

void Point::PutY(short Y) { this->Y = Y; }

void Point::PutC(size_t C) { this->C = C; }

short Point::GetX() {return X;}

short Point::GetY() {return Y;}

size_t Point::GetC() {return C;}

void Point::Show()
{
    HPEN Pen = CreatePen(PS_SOLID, 2, C);
    SelectObject(hdc, Pen); // сделать перо активным
    short R = 2, Ri;
    for (char i = -1; i < 2; i++) // цикл утолщения точки
    {
        Ri = R * i;
        Arc(hdc, X - Ri, Y - Ri, X + Ri, Y + Ri, X, Y + Ri, X, Y + Ri);
    }
    DeleteObject(Pen);
}
```

```

void Point::Hide()
{
    size_t TempColor = C;
    C = RGB(241, 241, 241);
    Show();
    C = TempColor;
}

void Point::MoveTo(short X, short Y)
{ Hide();
  PutX(X);
  PutY(Y);
  Show();
}

Line::Line(Point P1, Point P2, size_t C) // аргументы метода - объекты
{ this->P1 = P1; // присвоение объектов объектам
  this->P2 = P2;
  this->C = C;
}

void Line::Show()
{ HPEN Pen = CreatePen(PS_SOLID, 2, C);
  SelectObject(hdc, Pen); // сделать перо активным
  line(hdc, P1, P2);
  DeleteObject(Pen);
}

void Line::Hide()
{
    size_t TempColor = C;
    C = RGB(241, 241, 241);
    Show();
    C = TempColor;
}

void Line::Move(short DX, short DY)
{ Hide();
  P1.PutX(P1.GetX() + DX);
  P1.PutY(P1.GetY() + DY);
  P2.PutX(P2.GetX() + DX);
  P2.PutY(P2.GetY() + DY);
  Show();
}

void line(HDC hdc, Point P1, Point P2) // функция рисования отрезка
{
    MoveToEx(hdc, P1.GetX(), P1.GetY(), NULL);
    LineTo(hdc, P2.GetX(), P2.GetY());
}

```

Файл Composition.cpp:

```
        /* использование разработанных классов в основной программе */
#include <windows.h>
#include <iostream>
#include "PointLine.h"

HDC hdc; // объявление контекста устройства

int main()
{
    system("color f0"); // неяркий белый цвет консоли (светло-серый)
    HWND hwnd = GetConsoleWindow(); // создать графическое окно
    if (hwnd != NULL)
    {
        hdc = GetWindowDC(hwnd); // открыть графическое окно
        /* создание объектов классов Point и Line*/
        Point A(250, 200, RGB(0, 0, 255));
        Point B(250, 500, RGB(255, 0, 0));
        Point C(700, 160, RGB(0, 255, 255));
        Point D(300, 160, RGB(255, 255, 0));
        Line L1(A, B, RGB(0, 255, 0)); // объекты-точки:
        Line L2(C, D, RGB(0, 255, 0)); // начальные значения

        if (hdc != 0)
        {
            getchar();
            A.Show(); getchar();
            B.Show(); getchar();
            C.Show(); getchar();
            D.Show(); getchar();
            L1.Show(); getchar();
            L2.Show(); getchar();
            L1.Hide(); getchar();
            L2.Hide(); getchar();
            L1.Show(); getchar();
            L2.Show(); getchar();
            A.MoveTo(800, 450); getchar();
            B.MoveTo(400, 750); getchar();
            L1.Move(30, 70); getchar();
            L2.Move(20, 80); getchar();
        }
        CloseWindow(hwnd); // Закрывать графическое окно
    }
    return 0;
}
```

Задание для лабораторной работы № 6

Создать предлагаемую композицию классов, где поля классов есть объекты других классов, как требует вариант задания. Изобразить объекты таких классов. Вывести на экран координаты точки трёхмерной фигуры, которая находится в основании этой фигуры.

Часть 1

1. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«решётчатый квадрат» как массив параллельных объектов «отрезок»,
«куб» как массив параллельных объектов «решётчатый квадрат».
2. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«решётчатый квадрат» как массив параллельных объектов «отрезок»,
«четырёхугольная пирамида» из нескольких убывающими в размерах параллельно расположенных «решётчатых квадратов».
3. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«окружность» с центром в объекте «точка», и радиусом «отрезком»,
«цилиндр» как массив направляющих параллельных «отрезков» и с основаниями - параллельными «окружностями».
4. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«решётчатый параллелограмм» как массив параллельных «отрезков»,
«параллелепипед» как массив параллельных «решётчатых параллелограммов».
5. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«решётчатый ромб» как массив параллельных «отрезков»,
«ромбическая призма» как массив параллельных «решётчатых ромбов».
6. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«треугольник» как массив параллельных отрезков с убывающей длиной,
«тетраэдр» как массив параллельных «треугольников», последовательно убывающих в размерах до размеров точки.
7. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«треугольник» как массив параллельных отрезков с убывающей длиной,
«треугольная призма» как массив параллельных, убывающих в размерах «треугольников».

8. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«окружность» с объектом «точка» в центре, с радиусом – «отрезком»,
«цилиндр» как массив параллельных объектов «окружность».
9. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«окружность» с объектом «точка» в центре, с радиусом – «отрезком»,
«сфера» - массив повёрнутых вокруг общего диаметра «окружностей».
10. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«решётчатый прямоугольник» как массив параллельных «отрезков»,
«прямоугольная пирамида» - массив параллельных, убывающих в размерах «решётчатых прямоугольников».
11. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«прямоугольник» как массив параллельных «отрезков»,
«прямоугольный параллелепипед» как массив параллельных «прямоугольников».
12. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«эллипс» с центром - объектом «точка» и полуосями «отрезками»,
«эллипсоидный цилиндр» как массив параллельных «эллипсов»,
расположенных друг над другом.
13. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«эллипс» с центром - объектом «точка» и полуосями «отрезками»,
«эллипсоид» как массив «эллипсов», повёрнутых вокруг оси.
14. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«окружность» с объектом «точка» в центре, с радиусом – «отрезком»,
«конус» как массив параллельных убывающих «окружностей».
15. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«решётчатый ромб» как массив параллельных «отрезков»,
«ромбическая пирамида» - массив параллельных убывающих «ромбов».
16. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«эллипс» с центром, заданным объектом «точка»,
«эллипсоид» как массив параллельных согласованными «эллипсов».

17. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«окружность» с центром — объектом «точка» и радиусом — «отрезком»,
«конус» - массив «отрезков» с общей точкой, образующих конус.
18. «точка»,
«отрезок», определяемый двумя объектами «точка»,
«параллелограмм» - массив параллельных «отрезков»,
«пирамида с параллелограммом в основании» - массив параллельных,
убывающих в размерах «параллелограммов».

Часть 2

1. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«квадрат» Square, заданный двумя объектами Point или объектом Line,
«куб» Cub с двумя вершинами Point, или с ребром Line, или с гранью
Square.
2. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«квадрат» Square, заданный двумя объектами Point или объектом Line,
«четырёхугольная пирамида» Pyr основанием Square и вершиной Point.
3. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«окружность» Circle с радиусом Line,
«цилиндр» Cylinder, с основанием Circle и с высотой - отрезком Line.
4. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«параллелограмм» Parallelogram со стороной Line и с вершиной Point,
не принадлежащей этой стороне,
«параллелепипед» Parallelepiped с нижней гранью Parallelogram и с
вершиной Point, не принадлежащей этой грани.
5. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«ромб» Rhomb, вершины — «точки» Point, стороны — «отрезки» Line,
«призма» Prizma с вершинами Point, рёбрами Line, с основаниями
Rhomb и боковыми гранями Parallelogram.
6. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«треугольник» Triangle со стороной Line и с вершиной Point,
«тетраэдр» Tetraedr, задаваемый четырьмя вершинами Point или двумя
скрещивающимися рёбрами Line или с треугольной гранью Triangle и с
вершиной типа Point.

7. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
"треугольник" Triangle, заданный тремя "точками" Point или стороной
типа Line и вершиной типа Point,,
"треугольная призма" TrePrizma с треугольными основаниями Triangle и
с высотой H.
8. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
"окружность" Circle с радиусом — "отрезком" Line, один из концов Line
— центр окружности,
"цилиндр" Cylinder с основанием "окружность" и с высотой H.
9. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
"окружность" Circle с радиусом — "отрезком" Line, один из концов Line
— центр окружности,
"сфера" Sphera, изображаемая тремя окружностями с общим центром и
с одинаковым радиусом, "отрезком" Line, один из концов Line — центр
сферы.
10. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
"прямоугольник" Rectangle, со стороной основания Line и с длиной
боковой стороны b,
"пирамида с прямоугольным основанием" RectPyramida, задаваемая
вершиной Point и прямоугольным основаниями Rectangle.
11. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
"прямоугольник" Rectangle, со стороной основания Line и с длиной
боковой стороны b,
"прямоугольный параллелепипед" RectParallelepiped, задаваемый
прямоугольным основаниями Rectangle и высотой H.
12. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
"эллипс" Ellipse с центром Point и полуосями размерами a и b,
"эллиптический цилиндр", имеющий основание Ellipse и высоту H.
13. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
"эллипс" Ellipse с центром Point и полуосями размерами a и b,
"эллипсоид", изображаемый тремя объектами типа Ellipse с общим
центром и с попарно совпадающими полуосями a, b и c.

14. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«окружность» Circle с радиусом Line,
«конус» с основанием Circle и с вершиной Point.
15. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«ромб» Rhomb с диагональю Line и с величиной второй диагонали d,
«ромбическая пирамида» RhombPyramida с основанием Rhomb и с
вершиной Point.
16. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«эллипс» Ellipse с центром Point и полуосями размерами a и b,
«эллиптический конус» ElConus с основанием Ellipse и вершиной Point.
17. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«окружность» Circle с центром Point и с радиусом величиной R,
«конус» Conus с основанием Circle и вершиной Point.
18. «точка» Point,
«отрезок» Line, определяемый двумя объектами «точка» типа Point,
«параллелограмм» Parallelogram со стороной Line и с вершиной Point,
не принадлежащей этой стороне,
«пирамида с параллелограммом в основании» ParPyramida с основанием
Parallelogram и с вершиной Point.

Методические указания

Особенности композиции и понятие *композиция* и рассмотрены в [1]. Композиция естественна, как гласит девиз «Всё есть объект», главный для объектно-ориентированных программ. Естественность композиции есть причина того, что термин «композиция» не часто встречается в литературе. Связанный с понятием композиции термин *агрегация* рассмотрен в [8], а объекты, являющиеся полями данных, названы *подобъектами* в [6]. Необходимые при композиции на C++ конструкторы *по умолчанию* приведены в [10]. Композиция естественна, но при переходе от других парадигм программирования к объектно-ориентированной парадигме использование объектов в качестве полей весьма непривычно. Поэтому для лучшего освоения объектно-ориентированной парадигмы очень важны тренировки в композиции.

РАБОТА № 7. ПОЗДНЕЕ СВЯЗЫВАНИЕ. ДИНАМИЧЕСКИЕ ОБЪЕКТЫ

Цель работы - изучение особенностей создания и использования динамических объектов на Visual Studio C++ 2019|2022. Деструкторы для удаления динамических объектов и для удаления динамических полей в объектах в объектно-ориентированных программах. Особенности работы с динамически выделяемой памятью при позднем связывании.

Общие сведения

Позднее связывание, упомянутое в работе №4, называемое также **динамическое связывание** или **связывание во время исполнения (runtime)** состоит в том, что вычислительный процесс во время выполнения программы может быть перемещён из одного сегмента памяти в другой. Такое связывание реализуется на аппаратном уровне. В настоящее время при разработке пользовательских программ в большинстве программных систем, используется, главным образом, именно позднее связывание.

Как было изучено в работе №4, иллюстрацией позднего связывания являются виртуальные функции-члены, подключаемые к программе не на этапе компиляции, а на этапе выполнения. Однако этим примером приём подключения к программе на этапе выполнения не ограничивается. Другой разновидностью позднего связывания является динамически выделяемая оперативная память, известная в программировании с конца 50-х годов XX века. Динамически выделяемая память - оперативная память, подключаемая на этапе выполнения программы. Коротко её называют **динамическая память**. В противоположность динамической памяти оперативную память, выделяемую на этапе компиляции, называют памятью, выделяемой в стеке (реже статической памятью), подчёркивая аппаратный характер выделения. Часто в противоположность термину «*память, выделяемая в стеке*», динамическую память называют память, выделяемая в «куче». Это название подчёркивает, что ячейки оперативной памяти выделяются операционной системой не столь строго упорядоченно, как при выделении памяти в стеке. При динамическом выделении памяти необходимо кроме значений данных указанных типов хранить ещё и служебную информацию для операционной системы. Поэтому динамическая память требует при своём использовании несколько больших объёмов, чем при хранении таких же данных в стеке.

Идея динамической памяти как способа выделения оперативной памяти для работы программы изначально была предложена для того, чтобы экономить оперативную память. Считалось, что таким образом можно будет повторно использовать уже ненужную оперативную память в той же программе для решения новых задач. Соответствующие этому подходу динамические объекты [1, 3, 4] в объектно-ориентированных программах создаются также не на этапе компиляции, а на этапе выполнения. Когда динамический объект использован программой и не нужен, его можно удалить, и освободить занятую под этот объект оперативную память,

используя её во время работы программы для других целей. Этот способ активно используется в больших объектно-ориентированных программах со многими разнорезультативно используемыми объектами. Ведь предвидеть заранее объём требуемой оперативной памяти непросто, а порой и невозможно. В тоже время резервировать память заранее сразу под все программные ситуации может быть весьма затратно, даже если все программные ситуации удастся предвидеть. По этой причине динамическое выделение оперативной памяти является актуальной задачей, необходимой при создании современных программ, в том числе на Visual Studio C++ 2019/2022. Для решения этой задачи в языке C++ созданы две конструкции: конструкция **new** для динамического выделения, и конструкция **delete** для динамического освобождения.

Динамическое выделение памяти объекту выполняется следующим образом:

указатель_на_объект = new конструктор_класса (аргументы);

Видно, что для создания динамического объекта в C++ необходимо создать указатель на этот объект.

Динамическое освобождение памяти при удалении динамических объектов и в C++ выполняется синтаксической конструкцией **delete**:

delete *указатель_на_объект;*

Справедливости ради, важно заметить, что при выделении памяти в стеке также имеется возможность повторного использования оперативной памяти, которая состоит в том, что переменным, выполнившим свои задачи, присваивался другой смысл и новые значения. Это позволяет не создавать новые переменные и не выделять под них дополнительную оперативную память. Если в программе несколько циклов, расположенных друг за другом и использующих для счётчика одну и ту же переменную, то это очевидный пример такой ситуации.

При использовании динамической памяти предполагается не столь детерминированная ситуация, так как выделенная, а затем освобождённая оперативная память может быть использована при дальнейшем выполнении программы в другом месте, но без конкретного указания, где именно. Такая ситуация гораздо гибче, чем ситуация со стеком.

При фактической реализации в языках программирования оказалось, что технически неудобно и фактически невозможно, чтобы освобождённая оперативная память всякий раз по команде из программы «подбиралась» операционной системой. В результате вышеназванная конструкция **delete** стала не командой процессору на высоком уровне, как другие операторы языка C++, а превратилась в служебное информационное сообщения для операционной системы о том, что эта именованная область оперативной памяти, выделенная под объекты или под какие-либо другие переменные, уже не нужна. При этом совсем неважно, удаляется сам динамический

объект с помощью delete, или же с помощью delete освобождаются его динамические поля данных при вызове деструктора (см. ниже).

Повторный вызов delete для удаления того же самого объекта не представляет собой какой-либо проблемы, так как это лишь повторное сообщение от программы к операционной системе «Эта область памяти больше не нужна». В настоящей работе предлагается это проверить.

На самом деле освобождённая динамическая память может быть снова использована операционной системой в той же программе, где она была освобождена, если это будет необходимо для самой операционной системы. Например, если общий объём используемой во время работы программы оперативной памяти приближается к своему аппаратному пределу. При этом не следует принимать возможное искусственное обнуление числовых полей данных за удаление объекта, так как это всего лишь уничтожение информации, а не освобождение памяти. В небольших программах память, заявленная как ненужная, может никогда и не быть подключённой вновь к этой же программе из-за большого объёма свободной оперативной памяти. Однако динамическая память даже в программах, не требующих больших ресурсных затрат, может оказаться необходимой в случае, если до начала программы невозможно узнать, какой объём памяти понадобится.

Проблемы, связанные с динамическим освобождением памяти, привели к тому, что в языках Java и C#, появившихся после C++, отказались вовсе от команд, аналогичных delete, как от синтаксических конструкций. Для работы с освободившейся динамической памятью был создан *автоматический сборщик мусора* (garbage collector), с конца 50-х годов XX века известный по языку ЛИСП. Сейчас в языках Common Lisp, Java и C#, нет операторов удаления объектов, а есть автоматический сборщик мусора.

Использование конструкции new для выделения динамической памяти нуждается в комментариях. В C++ далеко не во всех версиях динамическая память выделяется сразу после действия вроде

```
pC = new Complex (5, 12);
```

Этому действию предшествует объявление указателя

```
Complex *pC;
```

Если после объявления указателя вывести на печать поля объекта, на который указывает pC, то современные версии C++ выдадут сообщение об ошибке или предупреждение. Возможная следующая ситуация:

```
Complex *pC;
```

```
...
```

```
pC->PrintComp(); // возможно сообщение об ошибке
```

```
...
```

```
pC = new Complex (5, 12);
```

Чтобы избежать этих проблем и несуразностей в C++ предлагается надёжная и однозначно трактуемая конструкция:

```
Complex * pC = new Complex (5, 12);
```

Теперь до этой строчки объекта нет. Запрос до этой строки об объекте, на который указывает pC, выдаст ошибку компилятора. А после этой строчки объект успешно существует с заданными полями. Запрос об объекте, на который указывает pC, будет успешно, однозначно и достоверно выполнен. Результат действия этой конструкции однозначен. Динамическая память выделяется в момент перехода программы к этой строке.

В языках Java и C# предлагается аналогичная конструкция:

Complex C = new Complex (5, 12);

(В этих языках уже не нужно создавать указатель: сразу создают объект C.) В примере 7.1 показано использование динамических объектов. При этом использован класс Complex, объявленный в файле Complex.h из примера 1.4 и определённый в файле Complex.cpp из примера 1.5. Динамические объекты в основной программе показаны в файле ComplexDyn.cpp аналогично статическим из примера 1.3. Новый проект состоит из файлов Complex.h, Complex.cpp и ComplexDyn.cpp.

Пример 7.1. Создание и удаление динамических объектов в основной программе на C++ для Visual Studio 2019/2022

Файл ComplexDyn.cpp с динамическими объектами в основной программе

```
#include <iostream>
#include <windows.h>
#include "Complex.h"

using namespace std;

int main()
{
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    system("color f0");
    Complex* pC;           // объявление указателя на объект типа Complex
    cout << "CPP: Dynamical Object" << endl;
    pC = new Complex(1, -1); // создание динамического объекта
    pC->PrintComplex();      getchar();
    cout<<"pC->Re=" << pC->GetRe() << " pC->Im=" << pC->GetIm() << "\n";
    getchar();
    pC->PutRe(3);
    pC->PutIm(4);
    cout << "Re=" << pC->GetRe() << " Im=" << pC->GetIm() << "\n";
    pC->PrintTrig();        getchar();
    delete pC; //удаление динамического объекта
    getchar();
    system("Pause");
    return 0;
}
```

Динамическое выделение оперативной памяти возможно в объектно-ориентированных программах не только тогда, когда создаётся объект, но и тогда, когда инициализируются поля данных объекта. В этом случае на C++ при объявлении класса вместо полей данных объявляются указатели на эти поля. Использование динамических полей показано в примере 7.2.

Пример 7.2. Использование динамических полей данных у динамических объектов при создании и удалении динамических объектов на C++

```
#include <iostream>
#include <windows.h>
#include <math.h>

using namespace std;

/* Объявление класса */
class Complex
{
private:
    double *pRe;
    double *pIm;
public:
    Complex(double, double);
    double GetRe();
    double GetIm();
    void PutRe(double);
    void PutIm(double);
    void PrintComplex();
    void PrintTrig();
    Complex Sum(Complex);
private:
    double Amp();
    double Fi();
};

Complex sum(Complex, Complex);

/* определение методов класса Complex */
Complex::Complex(double Re, double Im)
{
    pRe = new double;
    pIm = new double;
    (*pRe) = Re;
    (*pIm) = Im;
}

double Complex::GetRe() { return *pRe; }
double Complex::GetIm() { return *pIm; }
```

```

void Complex::PutRe(double Re) { *pRe = Re; }
void Complex::PutIm(double Im) { *pIm = Im; }
void Complex::PrintComplex()
    { cout << " Re = " << *pRe << " Im = " << *pIm << endl; }
double Complex::Amp() { return sqrt(*pRe * *pRe + *pIm * *pIm); }
double Complex::Fi() { return atan2(*pIm, *pRe); }
void Complex::PrintTrig() { cout<<"A = "<<Amp()<<" Fi = "<<Fi()<< endl; }
Complex Complex::Sum(Complex B)
{
    Complex C(0, 0);
    C.PutRe(*pRe + B.GetRe());
    C.PutIm(*pIm + B.GetIm());
    return C;
};

Complex sum(Complex A, Complex B)
{
    Complex C(0, 0);
    C.PutRe(A.GetRe() + B.GetRe());
    C.PutIm(A.GetIm() + B.GetIm());
    return C;
};

int main()
{
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    system("color f0");
    Complex* pC;
    cout << "CPP: Dynamical Fields" << endl;
    pC = new Complex(1, -1);
    pC->PrintComplex(); getchar();
    cout<<"pC->Re="<<pC->GetRe()<<" pC->Im="<<pC->GetIm()<<"\n";
    getchar();
    pC->PutRe(3);
    pC->PutIm(4);
    pC->PrintComplex(); getchar();
    pC->PrintTrig(); getchar();
    delete pC;  getchar();
    system("Pause");
    return 0;
}

```

При работе с динамическими полями данных для удаления объектов удобно использовать деструкторы.

Деструкторами называют методы, предназначенные для удаления объектов. Формально, по своему предназначению, методы-деструкторы противоположны конструкторам. Однако, фактически, необходимость в методах-деструкторах возникает только тогда, когда требуется удалить динамические поля данных, как в примере 7.2. В С++ имя деструктора создаётся на основе имени класса, перед которым ставится *тильда* ~. Так у класса Complex будет деструктор **~Complex()**. Тип возвращаемого значения у деструкторов, как и у конструкторов класса, не указывается. Но, в отличие от конструкторов, которые могут быть вызваны неявно, деструкторы на С++ всегда вызываются *явно*.

Деструкторы должны осуществлять удаление динамических полей данных, аналогично тому, как конструкторы создают динамические поля данных и задают начальные значения полей данных. Освободившаяся при удалении динамических полей данных оперативная память будет указана операционной системой как ненужная в этой программе. Так в примере 7.2 класс Complex с деструктором будет выглядеть следующим образом:

```
class Complex
{
private:
    double *pRe;
    double *pIm;
public:
    Complex(double, double);
    ~Complex();
    double GetRe();
    double GetIm();
    void PutRe(double);
    void PutIm(double);
    void PrintComp();
    void PrintTrig();
private:
    double Amp();
    double Fi();
};
```

Текст деструктора ~Complex() выглядит следующим образом:

```
Complex::~~Complex()
{
    delete pRe;
    delete pIm;
}
```

Деструктор для удаления динамического объекта используется так:

`pC->~Complex();`

Важно знать, что на Visual Studio C++ 2019/2022 нельзя для удаления динамического объекта одновременно использовать конструкцию `delete pC;` и конструкцию `pC->~Complex();`, так как будет выдана ошибка. Поэтому при динамических полях данных следует вызывать деструктор. Если же память для полей данных выделяется в стеке, то следует удалять динамический объект с помощью `delete`. Основная программа `main`, где показано использование деструктора `~Complex`, приведена в примере 7.1.

Пример 7.3. Использование деструкторов для удаления динамических объектов в основной программе на Visual Studio C++ 2019/2022

```
int main()
{
    SetConsoleOutputCP(1251);
    SetConsoleCP(1251);
    system("color f0");
    Complex * pC;
    pC= new Complex(1,-1);
    pC->PrintComp();
    getchar();
    cout<<"pC->Re="<<pC->GetRe()<<" pC->Im="<<pC->GetIm()<<"\n";
    getchar();
    pC->PutRe(3);
    pC->PutIm(4);
    pC->PrintComp();
    getchar();
    pC->PrintTrig();
    getchar();
    pC->~Complex();
    system("Pause");
    return 0;
}
```

Результаты работы программы из примера 7.3:

CPP: Dynamical Fields

Re = 1 Im = -1

pC->Re=1 pC->Im=-1

Re = 3 Im = 4

A = 5 Fi = 0.927295

Динамические и полиморфные объекты в Visual Studio C++ 2019/2022

В работе №5 рассмотрены полиморфные объекты, меняющие свой тип во время работы программы. В настоящей работе изучены динамические объекты, создаваемые и удаляемые во время работы программы. Эти возможности работы с объектами на Visual Studio C++ 2019/2022 могут быть объединены. Другими словами, на языке C++ возможны: неполиморфные статические объекты, полиморфные статические объекты, неполиморфные динамические объекты и полиморфные динамические объекты. Иногда в литературе соединение полиморфизма объектов с динамически выделяемой памятью для таких объектов называется **динамический полиморфизм** [4]. Пример создания полиморфных динамических объектов рассмотрен ниже.

В примере 7.4 показано использование динамических и полиморфных объектами. Для выполнения этого объявлены указатели на объекты классов Point и Circle. С помощью конструкции new выделена динамическая память под созданные объекты. Затем вызываются функции Clear и Move из примера 5.2 с полиморфными аргументами. После этого создается указатель pL на абстрактный класс Location следующим образом: Location* pL; Этот созданный указатель pL последовательно полиморфно превращается в указатели на объекты классов Point и Circle и вызывается функциями Clear и Move. При этом вызовы по форме не изменяют аргумент pL: Clear(pL); или вроде Move(360, 700, pL);, в то время как по содержанию аргумент pL меняется. По окончании использования динамических полиморфных указателей на объекты динамическая память освобождается с помощью конструкции delete.

Пример 7.4. Работа с полиморфными динамическими объектами

```
#include <windows.h>
#include <iostream>
#include "PointCircle.h"

HDC hdc; // объявление контекста устройства

int main()
{
    system("color f0"); // цвет консоли - неяркий белый (светло-серый)
    HWND hwnd = GetConsoleWindow(); // создать графическое окно
    if (hwnd != NULL)
    {
        hdc = GetWindowDC(hwnd); // открыть графическое окно
        /* создание динамических объектов классов Point и Circle */
        Point* pP1 = new Point(250, 200, RGB(0, 0, 255));
        Point* pP2 = new Point(800, 160, RGB(255, 0, 0));
        Circle* pC1 = new Circle(250, 500, 50, RGB(0, 255, 0));
        Circle* pC2 = new Circle(800, 160, 70, RGB(255, 0, 0));
        if (hdc != 0)
```

```

{
    getchar();
    pP1->Show(); getchar();
    pP2->Show(); getchar();
    pC1->Show(); getchar();
    pC2->Show();
    getchar(); Clear(pP1); //вызов функции с полиморфным аргументом
    getchar(); Clear(pP2); //вызов функции с полиморфным аргументом
    getchar(); Clear(pC1); //вызов функции с полиморфным аргументом
    getchar(); Clear(pC2); //вызов функции с полиморфным аргументом
    getchar();
    pP1->Show(); getchar();
    pP2->Show(); getchar();
    pC1->Show(); getchar();
    pC2->Show();
    getchar(); Move(360, 700, pP1); // полиморфный аргумент у функции
    getchar(); Move(230, 820, pP2); // полиморфный аргумент у функции
    getchar(); Move(610, 120, pC1); // полиморфный аргумент у функции
    getchar(); Move(420, 250, pC2); // полиморфный аргумент у функции
    Location* pL;          // указатель на абстрактный объект
    pL = pP1;              // полиморфный указатель
    Clear(pL); getchar();
    pL->Show(); getchar();
    Move(600, 750, pL); getchar();
    pL = pP2;              // полиморфный указатель
    Clear(pL); getchar();
    pL->Show(); getchar();
    Move(420, 500, pL); getchar();
    pL = pC1;              // полиморфный указатель
    Clear(pL); getchar();
    pL->Show(); getchar();
    Move(330, 570, pL); getchar();
    pL = pC2;              // полиморфный указатель
    Clear(pL); getchar();
    pL->Show(); getchar();
    Move(160, 270, pL); getchar();
}
delete pP1;              // освобождение динамической памяти
delete pP2;              // освобождение динамической памяти
delete pC1;              // освобождение динамической памяти
delete pC2;              // освобождение динамической памяти
CloseWindow(hwnd); // закрыть графическое окно
}
return 0;
}

```

В примере 7.4 показан только текст одной функции main. Объявления как классов Location, Point и Circle, так и функций Clear и Move, а также определения методов классов и определения функций в проекте с такой функцией main должно быть такими же как в примере 5.1 и примере 5.2.

Очень важно строго усвоить непростую терминологию, связанную с динамическим выделением оперативной памяти объектам. В Интернете ещё порой встречается определение термина «динамический полиморфизм» как «механизм, при котором одна и та же инструкция может быть использована для работы с объектами разных типов, но конкретный тип и инструкции не известны на этапе компиляции, а определяются на этапе выполнения». Анализ этого определения показывает, что оно подходит для программных ситуаций, связанных с поздним связыванием и рассмотренных в работе №4, и в текущей работе №7. Однако для работы с виртуальными функциями там, где используется полиморфизм методов (переопределение), вовсе не нужно создавать динамические объекты (как показано в работе №4). При этом динамическое выделение памяти для объектов (как показано в настоящей работе), не обязательно предполагает полиморфизм объектов. С этой точки зрения термину *динамический полиморфизм* следует придать более строгую трактовку, разделяя полиморфизмы (методов и объектов) и динамическое выделение оперативной памяти объектам при выполнении программы.

Задание для лабораторной работы № 7

1. Создать динамические объекты, используя классы, синтезированные при выполнении работы №5, в соответствии с заданием. Показать движение динамических объектов. Удалить динамические объекты.
2. Создав динамические поля данных, видоизменить классы так, как в примере 7.2. Выполнив операции с объектами классов с динамически создаваемыми полями данных, удалить объекты этих классов.
3. Добавить к новому классу деструктор, удаляющий динамические поля данных, аналогично примерам 7.2 и 7.3. После демонстрации движения этих объектов, удалить все объекты.

Отразить результаты исследования функционирования динамических объектов, динамических полей данных и деструкторов в выводах к работе. Высказать собственные соображения по поводу эффективности динамического освобождения динамически выделенной оперативной памяти непосредственно по команде пользователя в исходном тексте прикладной программы. Рассмотреть и обсудить эффективность создания и использования деструкторов класса.

Методические указания

При подготовке к выполнению задания следует ознакомиться с понятием *динамическая память* и со способами её выделения и освобождения, а также с понятием *деструктор*, рассмотренными, например, в [1, 3, 4].

ДОПОЛНИТЕЛЬНЫЕ ВОЗМОЖНОСТИ ЯЗЫКА Visual Studio C++ 2019/2022 В ОБЪЕКТНО-ОРИЕНТИРОВАННЫХ ПРОГРАММАХ

Связь между классами в объектно-ориентированных программах может быть организована тремя способами. Первый из них - наследование, второй - композиция. Третьим способом взаимодействия классов является использование классов как библиотек функций-членов. В этом случае под именем одного и того же класса объединяются несколько функций-членов, выполняющих схожие задачи. При желании программиста использовать эти функции-члены в других классах или в основной программе, вызывать их можно с помощью конструкции:

<имя_класса>::<имя_метода>.

Например, в одном классе Sredni собраны несколько функций-членов для вычисления средних значений двух вещественных чисел: среднего арифметического Arif, среднего геометрического Geom, среднего квадратического Quadr, среднего гармонического Garm. Помимо этих методов в классе Sredni могут быть другие переменные-члены и функции-члены для решения иных программных задач. Однако, если необходимо использовать эти функции-члены самостоятельно, в теле методов других классов или в основной программе, их следует вызывать как;

Sredni::Arif(x, y)
или Sredni::Geom(3, 27)
или Sredni::Quadr(a, b)
или Sredni::Garm(40, 60).

Организация вычислений в этом случае позволяет обращаться к функциям-членам класса без создания объектов этого класса, напрямую.

Конструкция «двойное двоеточие» :: может быть использована также тогда, когда имена глобальной переменной и переменной-члена совпадают. Например, глобальная переменная объявляется как `int i;` и переменная-член объявляется `int i;` совпадая по форме. Тогда обращение из тела функции-члена к глобальной переменной должно быть ::i, в то время как к полю с тем же именем нужно обращаться просто i.

В языке C++ имеется возможность создавать классы в теле других классов. Такие классы называют *вложенные классы* [3]. Если внутри класса А есть вложенный класс В, то объекты класса В могут существовать строго только внутри класса А.

Кроме возможности создавать класс в теле другого класса, язык C++ обладает ещё и возможностью создавать класс в теле функции. Эти классы называют *локальные классы* [3]. Если внутри функции f есть локальный класс С, то объекты класса С могут существовать только внутри функции f.

В языке C++ существует возможность *множественного наследования* классов, когда один класс-наследник имеет два и более базовых класса. Но при использовании множественного наследования может возникнуть опасность неоднозначной работы компилятора: ромбовидное наследование.

Ромбовидное наследование иллюстрирует пример: базовый класс А имеет метод m() и классы-наследники В и С. В классах-наследниках В и С метод m() переопределяется, в каждом классе по-своему [3]. Класс D наследует, согласно механизму множественного наследования, базовым классам В и С. Метод m() наследуется классом D и от класса В, и от класса С. Невозможно достоверно определить, какую из переопределённых версий метода m() в классе D подключит компилятор. Иерархия из этих четырёх классов внешне выглядит как ромб, потому такое наследование и называется ромбовидным. Для преодоления такой опасности ромбовидного наследования в языке C++ предлагается применять *виртуальные классы* [3] (позднее связывание).

Классы объявляются виртуальными для преодоления ромбовидного наследования в названном выше примере следующим образом [3]:

```
class A {...};
class B: public virtual A {...};
class C: public virtual A {...};
class D: public B, public C {...};
```

Тогда при выполнении программы будет подгружена только одна версия метода m(), в зависимости от программной ситуации.

В языке C++ существует уникальная особенность, отсутствующая в других объектно-ориентированных языках - дружественные функции и дружественные классы (friend). Функции, объявленной дружественной к классу, доступны закрытые компоненты этого класса. При этом функция, объявленная как дружественная к классу, не является функцией-членом этого класса. Если же класс объявляется дружественным (friend) к другому классу, то методам этого класса доступны закрытые компоненты класса, по отношению к которому он объявлен дружественным.

Наряду с этими особенностями языка C++ следует сказать о редкой возможности C++, называемой **перегрузка операций**. Операциям языка C++, таким как + - * / = >> << == & | && || ! ~ ++ -- += -= *= /= [] () и т.п. можно присвоить другой смысл, перегрузив их с помощью operator:

```
тип operator операция ( список параметров ) { тело функции }
```

Нельзя перегружать в C++ только операции . .* :: ?: # ## sizeof

Кроме указателей (pointer), в языке C++ широко используются ссылки (reference). Если указатель – это адрес объекта в оперативной памяти, то ссылка – это другое имя переменной. Объявление ссылок по своей форме отличается от объявления указателей тем, что вместо * используется символ &. При этом передача аргументов внутрь функции по ссылке, как и передача по указателю, позволяет избежать копирования значений аргументов, и даёт возможность вернуть изменённые аргументы в вызывающую программу.

В заключение важно заметить, что настоящее учебное пособие нужно для знакомства с объектно-ориентированным программированием на C++ в среде Visual Studio 2019/2022. Оно продолжает материал, изложенный в [10, 11]. Однако ограниченный издательским стандартом объём пособия не дал возможности рассмотреть всё богатство C++ для Visual Studio 2019/2022.

Библиографический список

1. Бадд Т. Объектно-ориентированное программирование в действии. – СПб.: Питер, 1997. – 464 с.
2. Намиот Д.Е. Язык программирования Turbo C++. – М.: Изд-во МГУ, 1991. – 121 с.
3. Березин Б.И., Березин С.Б. Начальный курс С и С++. – М.: ДИАЛОГ-МИФИ, 2005. – 288 с.
4. Шилдт Г. Самоучитель С++: Пер. с англ. – 3-е изд. – СПб.: БХВ-Петербург, 2001. – 688 с.
5. Шамис В.А. Borland C++Builder. Программирование на С++ без проблем. – М.: Нолидж, 1997. – 266 с.
6. Фэйсон Т. Объектно-ориентированное программирование на Borland C++ 4.5. – Киев: Диалектика, 1996. – 544 с.
7. Павловская Т.А. С/С++ Программирование на языке высокого уровня. – СПб.: Питер, 2001. – 464 с..
8. Павловская Т.А., Щупак Ю.А. С/С++ Структурное программирование. Практикум. – СПб.: Питер, 2007. – 239 с.
9. Новиков П.В. Увеличение объёма используемой оперативной памяти компьютера при наследовании классов в объектно-ориентированном программировании // Современная наука: актуальные проблемы теории и практики. Серия «Естественные и технические науки». 2018. №6, июнь. Раздел «Информатика, вычислительная техника и управление». С. 116-122.
10. Новиков П.В. Объектно-ориентированное программирование. – М.: Изд-во МАИ, 2007. – 49 с.
11. Новиков П.В. Объектно-ориентированное программирование / Под ред. проф., д.т.н. О.М. Брехова – М.: Изд-во МАИ, 2019. – 124 с.
12. Новиков П.В. Особенности использования виртуальных функций в объектно-ориентированных программах // Современная наука: актуальные проблемы теории и практики. Серия «Естественные и технические науки». 2021. №10, октябрь. Раздел «Информатика, вычислительная техника и управление». С. 96-102.

СОДЕРЖАНИЕ

Предисловие.....	3
Общий порядок выполнения лабораторных работ.....	4
Содержание отчётов по лабораторным работам.....	4
Р А Б О Т А № 1. ОБЪЕКТ И КЛАСС. ИНКАПСУЛЯЦИЯ	
Общие сведения.....	5
Задание для лабораторной работы № 1.....	24
Р А Б О Т А № 2. НАСЛЕДОВАНИЕ КЛАССОВ НА C++ ДЛЯ VISUAL STUDIO 2019/2022	
Общие сведения.....	26
Задание для лабораторной работы № 2.....	47
Р А Б О Т А № 3. ПОЛИМОРФИЗМЫ. ПЕРЕГРУЗКА И ПЕРЕОПРЕДЕЛЕНИЕ МЕТОДОВ	
Общие сведения.....	48
Задание для лабораторной работы № 3.....	54
Р А Б О Т А № 4. ПОЗДНЕЕ СВЯЗЫВАНИЕ. ВИРТУАЛЬНЫЕ ФУНКЦИИ-ЧЛЕНЫ	
Общие сведения.....	55
Задание для лабораторной работы № 4.....	60
Р А Б О Т А № 5. АБСТРАКТНЫЕ КЛАССЫ И ПОЛИМОРФНЫЕ ОБЪЕКТЫ. ИНТЕРФЕЙСЫ (interface) НА Visual Studio C++ 2019/2022	
Общие сведения.....	61
Задание для лабораторной работы № 5.....	70
Р А Б О Т А № 6. МЕТОД КОМПОЗИЦИИ. ПОДОБЪЕКТЫ	
Общие сведения.....	71
Задание для лабораторной работы № 6.....	76
Р А Б О Т А № 7. ПОЗДНЕЕ СВЯЗЫВАНИЕ. ДИНАМИЧЕСКИЕ ОБЪЕКТЫ	
Общие сведения.....	81
Задание для лабораторной работы № 7.....	89
ДОПОЛНИТЕЛЬНЫЕ ВОЗМОЖНОСТИ ЯЗЫКА Visual Studio C++ 2019/2022 В ОБЪЕКТНО-ОРИЕНТИРОВАННЫХ ПРОГРАММАХ...	
92	
Библиографический список.....	94

Тем. план 2025, ч. , поз.

Новиков Павел Владимирович

**ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ
ПРОГРАММИРОВАНИЕ**

Редактор

Компьютерная вёрстка *П.В. Новикова*

Подписано в печать ..2024.

Бумага писчая. Формат 60х84 1/16. Печать офсетная.

Усл. печ. л. 6,0. Уч.-изд. л. 6,0. Тираж 100 экз.

Заказ /.

Издательство МАИ
(МАИ), Волоколамское ш., д. 4,
Москва, А-80, ГСП-3 125993

Отпечатано с готового оригинал-макета
Типография Издательства МАИ
(МАИ), Волоколамское ш., д. 4,
Москва, А-80, ГСП-3 125993